CS 530 Lab Notebook

Lab 7

Name: Jarren Calizo
OdinID: calizo
Date: November 16, 2025

Table of Contents

Lab Overview 6
Lab 7.1a Terraform AWS Guestbook 7
Step 1. Terraform Overview 7
Step 2. Setup in AWS CloudShell 8
Step 3. Initial Configuration 9
Step 4. Launching configuration 9
1. Take a screenshot showing the completion of the command including its output 11
2. Take a screenshot that includes the VM's IP addresses 12
Step 5. Adding Network Access 12
Step 6. Adding ssh access 12
1. Take a screenshot of the successful ssh login from Cloud Shell. 13
Step 7. Adding the Guestbook application 14
1. Take a screenshot of the output of the command that includes the IP address of the instance 14
Step 8. View the Guestbook 15
1. Take a screenshot of the Guestbook including the URL with the entry in it. 15
Step 9. Clean Up 15
Lab 7.1g Terraform GCP Guestbook 16
Step 1. Terraform 16
Step 2. Setup 16
Step 3. Initial Configuration 16
Step 4. Launching Configuration 17
1. Take a screenshot that includes the VM's IP addresses 18
Step 5. Adding an external IP address 18
1. Take a screenshot showing the completion of the command including its output 19
2. Take a screenshot that includes the VM's IP addresses 20
Step 6. Adding ssh access 20
1. Take a screenshot of the successful ssh login from Cloud Shell. 21
Step 7. Adding the Guestbook application 21
1. What resources are being added, changed, or destroyed? 23
2. What part of the configuration forces a replacement to occur? 23
Step 8. View the Guestbook 24
1. Take a screenshot of the Guestbook including the URL with the entry in

it. 24
Step 9. Clean Up 24
Lab 7.2g Kubernetes Guestbook 25
Step 1. Kubernetes Overview 25
Step 2. Setup 25
Step 3. Assigning Privileges 25
Step 4. Create Kubernetes cluster 26
1. What is the name of the Instance Template dynamically generated to create the two nodes (VMs)? 27
2. What is the name of the Instance Group dynamically generated that the two nodes belong to? 27
3. What are the names of the two nodes? 27
Step 5. Prepare a container image 28
1. Take a screenshot of the container image created 28
Step 6. kubernetes.yaml 28
Step 7. Deploy the configuration 29
1. Take a screenshot of the output of the following command when all 3 replicas reach a "Running" state. 30
2. Take a screenshot of listing services with LoadBalancer indicating an external IP address that is ready for access. 31
Step 8. View the Guestbook 31
1. Take a screenshot of the Guestbook including the URL with the entry in it. 32
2. Take a screenshot of the managed guestbook pods and the service being exposed. 33
3. Take a screenshot of the load balancer and its details 34
4. Take a screenshot of the addresses allocated and indicate the ones associated with nodes versus the one associated with the load balancer. 35
Step 9. Delete workload and service 35
Step 10. CI/CD Build Automation 36
Step 11. Configure build automation 36
Step 12. Deploy and view application 36
1. Take a screenshot of the Guestbook including the URL with the entry in it. 37
Step 13. Clean up 37
Lab 7.3g APIs (Slack, Knowledge Graph) 38
Step 2. Code 38
1. Does Google provide a Python package specifically for accessing the Knowledge Graph API? 38
Step 3. Code 38

1. Show the source line that constructs the query we wish to send to the Knowledge Graph API. 39
2. Show the source line that then executes the query and saves the response. What is the name of the method that sends the query to the Knowledge Graph API? 39
3. What is the Python data type that is used to represent the formatted message? 39
4. What are the three main attributes of the formatted message passed back to Slack? 40
Step 4. Knowledge Graph API Configuration 40
Step 5. Slack Workspace and Signing Secret 41
Step 6. Deploying the Cloud Function 41
Step 7. Creating the Slack Command 42
Step 8. Test the command 42
1. Take a screenshot of its response for your lab notebook. 43
Lab 7.4 ML APIs 44
Step 1. Setting up the ML APIs 44
Step 2. IAM Service Account Setup 44
Step 3. Vision 44
1. Show the output for your lab notebook 45
2. What is the name of the function? 45
3. What type of Vision client is instantiated in it? 45
4. What method is invoked in the Vision client to perform the detection? 45
5. What is the name of the attribute in the response object that contains the results we seek? 45
6. Take a screenshot of the output for the above commands 46
7. What method is invoked in the Vision client to perform the detection? 46
Step 4. Speech 47
1. Show the output for your lab notebook 47
2. What is the name of the function? 47
3. What method is invoked in the Speech client to perform the detection? 48
4. What is the name of the attribute in the response object that contains the results we seek? 48
Step 5. Translate 48
1. Show the output for your lab notebook 48
2. What is the name of the function? 49
3. What method is invoked in the Translate client to perform the detection? 49
4. What is the name of the attribute in the response object that contains

the results we seek? 49
Step 6. Natural Language 50
1. Show the output for your lab notebook 50
Step 7. Integration Concept 50
Step 8. Code 51
1. What is the name of the function that performs the transcription? 51
2. What is the name of the function that performs the translation? 51
3. What is the name of the function that performs the entity analysis on the translation? 52
4. What is the name of the function that performs the entity analysis on the image? 52
Step 9. Test integration 52
1. If the program deems them unrelated, then based on the results from the APIs, what must be changed in the program to address this? 53
2. If the program deems them unrelated, then based on the results from the APIs, what must be changed in the program to address this? 53
3. If the program deems them unrelated, then based on the results from the APIs, what must be changed in the program to address this? 54
Step 10. Video Intelligence API 54
Step 11. Video Setup 54
Step 12. Running labels.py 54
Step 13. Video Intelligence 54
1. What are the 3 labels with the highest confidence that the Video Intelligence API associates with the video and what are the confidences for each? 55
2. What is the name of the client class in the package that is used? 56
3. What method is used in that class to perform the annotation? 56
Step 14. APIs #3 (Web-site Integration) 56
Step 15. IAM Service Account Setup 56
Step 16. Application 57
1. Take a screenshot for your lab notebook that includes the URL. 57
Step 17. Code 57
1. What line of code creates the query for previous detections? 58
2. What line of code sends the query to Cloud Datastore? 58
3. Show the line that retrieves the name of the storage bucket to use. 58
4. What form field is used to specify the uploaded photo? 58
5. Show the line that copies the photo's contents to the storage bucket. 59
6. What method in Vision's annotation client is used to perform the analysis? 59

7. What fields are stored in Cloud Datastore for each image? 59
8. What happens at the end of the upload_photo route? 60
Step 18. Clean Up 60

Lab Overview

Lab 7 was the first time this course truly felt like stepping into the wide, breathing ecosystem of cloud systems rather than simply configuring isolated machines. Each section pulled me deeper into the idea that cloud infrastructure is not just deployed, but orchestrated; not just provisioned, but taught to respond, scale, and communicate. Across Terraform, Kubernetes, Slack integrations, and Google’s ML APIs, the lab became a mosaic of modern cloud design, showing how services weave together into something larger than their parts.

The journey began with Terraform on AWS and GCP, a quiet but powerful reminder that infrastructure is ultimately a story written in configuration files. Declaring a VM, watching it materialize, expanding its abilities through security groups, keys, metadata, and startup scripts - all of it reinforced a truth about Infrastructure-as-Code: the cloud becomes predictable once you describe it honestly. Destroying everything at the end felt almost ceremonial, like closing the covers on a world I had built and unbuilt with intention.

Kubernetes expanded that world dramatically. Deploying the Guestbook through three replicas across two nodes made the application feel alive in a way that a single VM never could. Seeing pods reschedule, LoadBalancer IPs surface, and Datastore carry state across stateless replicas crystallized the beauty of container orchestration. Even creating the CI/CD automation with Cloud Build felt like gifting the cluster its own ability to grow - a simple commit evolving into a full deployment with no manual intervention.

The APIs labs shifted the focus from systems to intelligence. Slack’s slash command integration showed the precision of secure request verification and event-driven cloud functions. Tying Slack to Google’s Knowledge Graph made the communication feel almost conversational; I could feel the stitching of two ecosystems meeting through clean JSON and trusted signatures. The ML APIs - Vision, Speech, Translate, and Natural Language - felt like a toolbox of perception. Each call demonstrated a different edge of machine understanding, and combining them into a multi-stage integration made the system feel almost human: listening, translating, analyzing, and comparing.

The final Vision-Datastore-Web integration tied everything together in a full web pipeline. Uploading an image, streaming it through Cloud Storage, annotating it with Vision, storing results in Datastore, and reflecting it all in a Flask UI felt like watching the full arc of cloud-native development unfold. Every layer had a purpose: storage, computation, identity, logging, and intelligence.

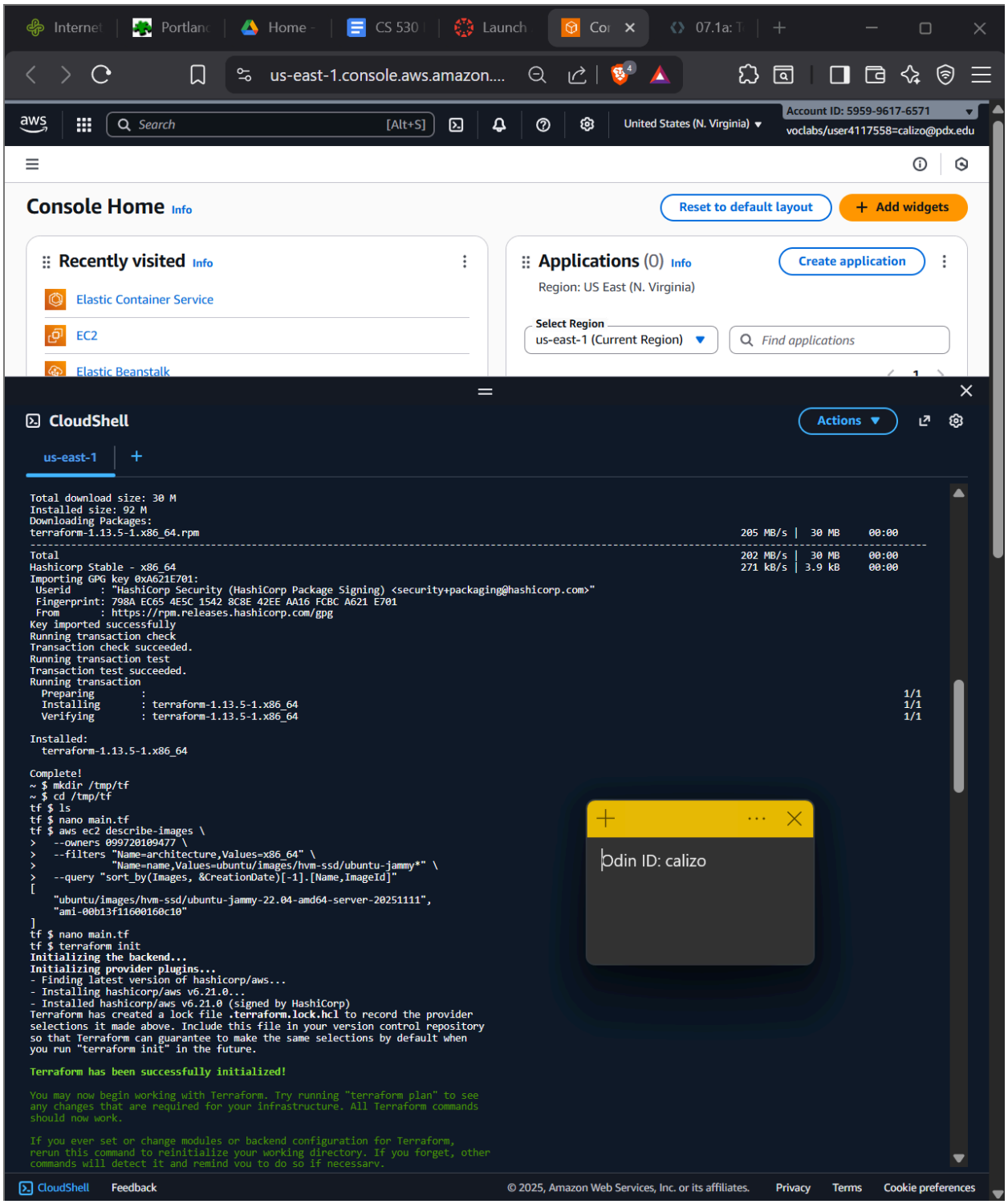
If Lab 6 taught me how cloud components communicate, Lab 7 taught me how they collaborate. Terraform built the stage, Kubernetes managed the cast, Cloud Build automated the choreography, Slack and Knowledge Graph introduced dialogue, and the ML APIs added perception. Together, they formed a single symphony of cloud engineering - declarative, scalable, event-driven, and intelligent. This lab wasn’t just a set of tasks; it was a glimpse into how modern systems breathe.

Lab 7.1a Terraform AWS Guestbook

Step 1. Terraform Overview

Before touching any configuration, I grounded myself in the rhythm of Terraform itself. The workflow feels a lot like gently laying out the architecture of a future world: declare the resources, preview the consequences, then let Terraform bring the environment to life. The lecture slides framed this philosophy well, and seeing it in practice here helped anchor the mindset for the rest of this lab. No commands were executed yet. This phase was conceptual.

Step 2. Setup in AWS CloudShell



I launched AWS CloudShell in us-east-1 and installed Terraform into the ephemeral environment. The installation sequence completed cleanly, as shown in my first screenshot. Once Terraform was ready, I created the working directory /tmp/tf and

moved into it. This step served as the foundation for everything that followed: a clean shell, the correct region, and the tooling prepared to orchestrate cloud resources.

Step 3. Initial Configuration

3a. Provider Block
Inside /tmp/tf, I created main.tf and added the AWS provider block, specifying us-east-1 as the deployment region. This anchors Terraform to the correct cloud backend and region.

3b. Locating the Ubuntu AMI
Next, I queried AWS for the most recent Ubuntu 22.04 LTS (Jammy) AMI. The screenshot shows the command returning the correct latest image. I extracted the AMI ID and used it when constructing the EC2 instance block.

3c. Defining the EC2 Instance and Output
Back in main.tf, I added the EC2 resource definition using:

- the retrieved AMI,
- the t2.micro machine type,
- and an output block configured to return the public IPv4 address of the VM.

This completed the minimal Terraform blueprint for launching our Guestbook host.

Step 4. Launching configuration

4a. terraform init
Running terraform init successfully fetched the AWS provider and initialized the working directory. This moment marks when Terraform becomes aware of the infrastructure we intend to create. The screenshot shows the initialization completing without warnings or errors.

4b. terraform plan

I ran terraform plan to preview the deployment. Terraform reported a single resource to be created: the EC2 instance. The screenshot captures this plan clearly, confirming the configuration was defined correctly.

4c. terraform apply

After verifying the plan, I executed terraform apply and approved the operation by typing "yes". Terraform created the EC2 instance and emitted the VM's public IP in the output block. The third screenshot shows this exact moment, with the instance fully provisioned and the output printed at the end of the apply process.

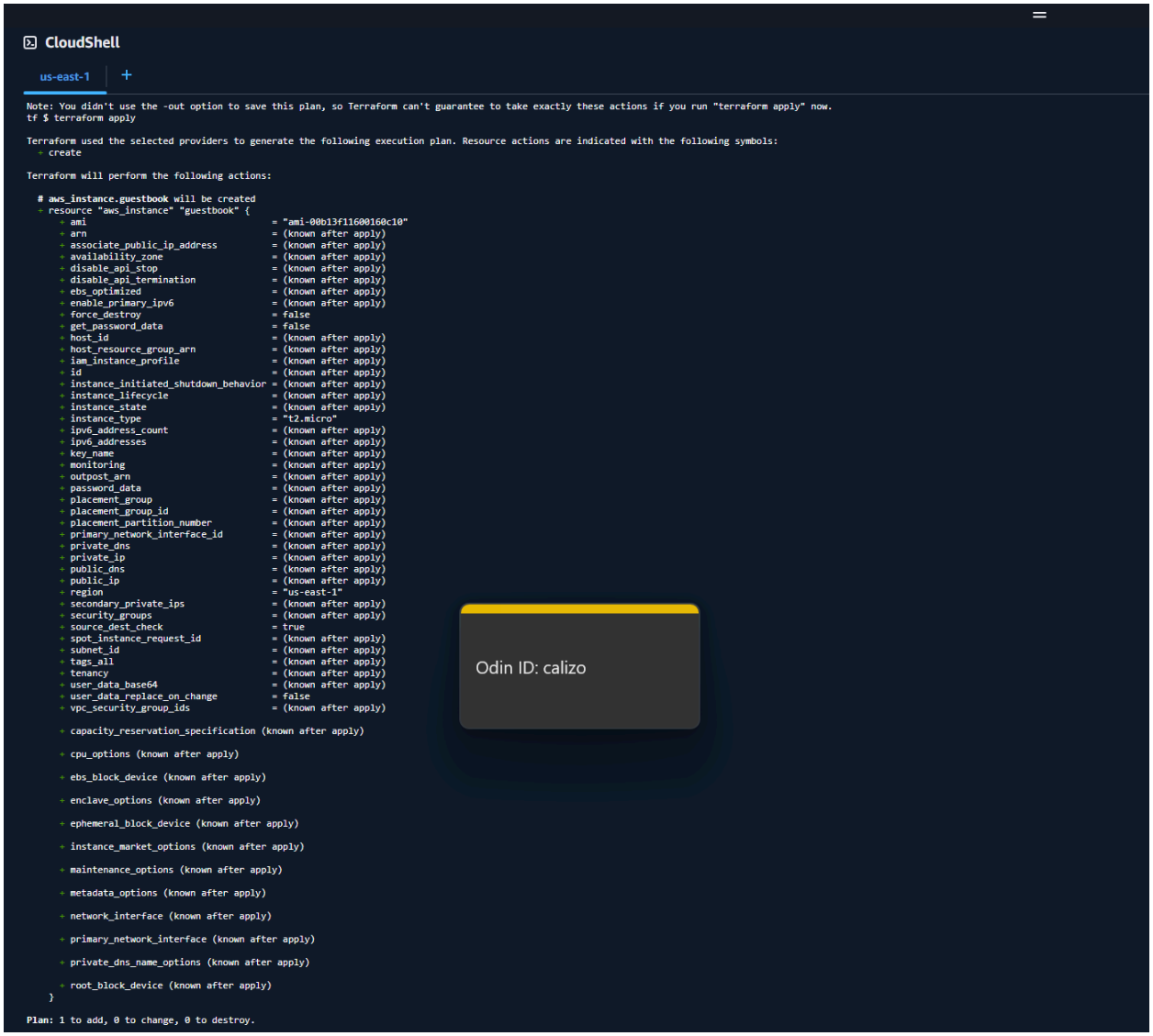
4d. Verifying in the EC2 Console

I then navigated to the EC2 Instances page in the AWS console to visually confirm the deployment. The instance appeared in the us-east-1 region, showing:

- Instance state: Running
- Instance type: t2.micro
- Public IPv4 address
- Private IPv4 address

The final screenshot confirms that the instance is active and reachable, completing this phase of the lab.

1. Take a screenshot showing the completion of the command including its output

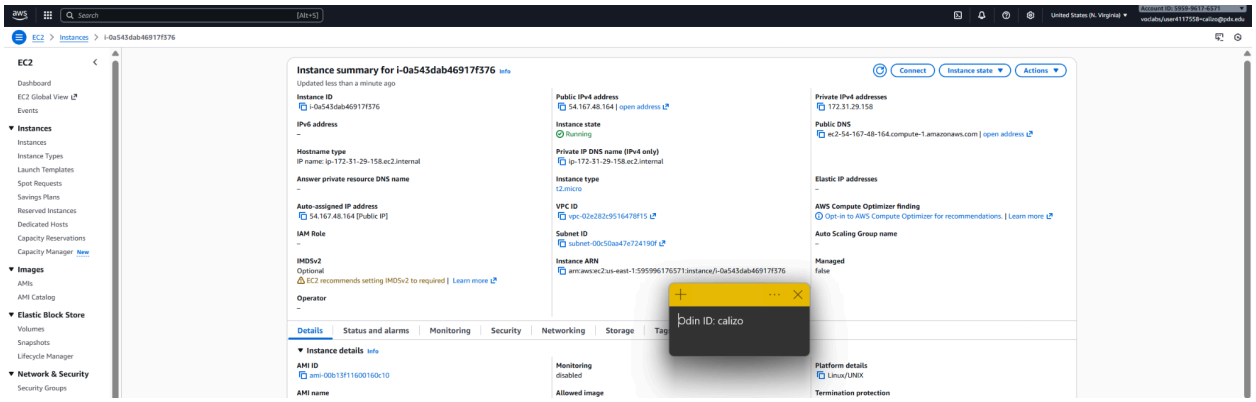




This shows Terraform reporting:

- Apply complete
- 1 resource added
- Output block displaying the assigned public IP

2. Take a screenshot that includes the VM's IP addresses



This screenshot is from the EC2 console and where both the public and private IPv4 addresses are shown clearly in the instance detail pane.

Step 5. Adding Network Access

As the Guestbook environment grew beyond a simple instance definition, this step asked me to open the doors for the outside world to reach the VM. I added a new aws_security_group block in main.tf, carving out explicit ingress routes for SSH on port 22 and HTTP on port 80. This small declaration felt like opening a controlled passage into the machine, allowing both administrator entry and future web traffic.

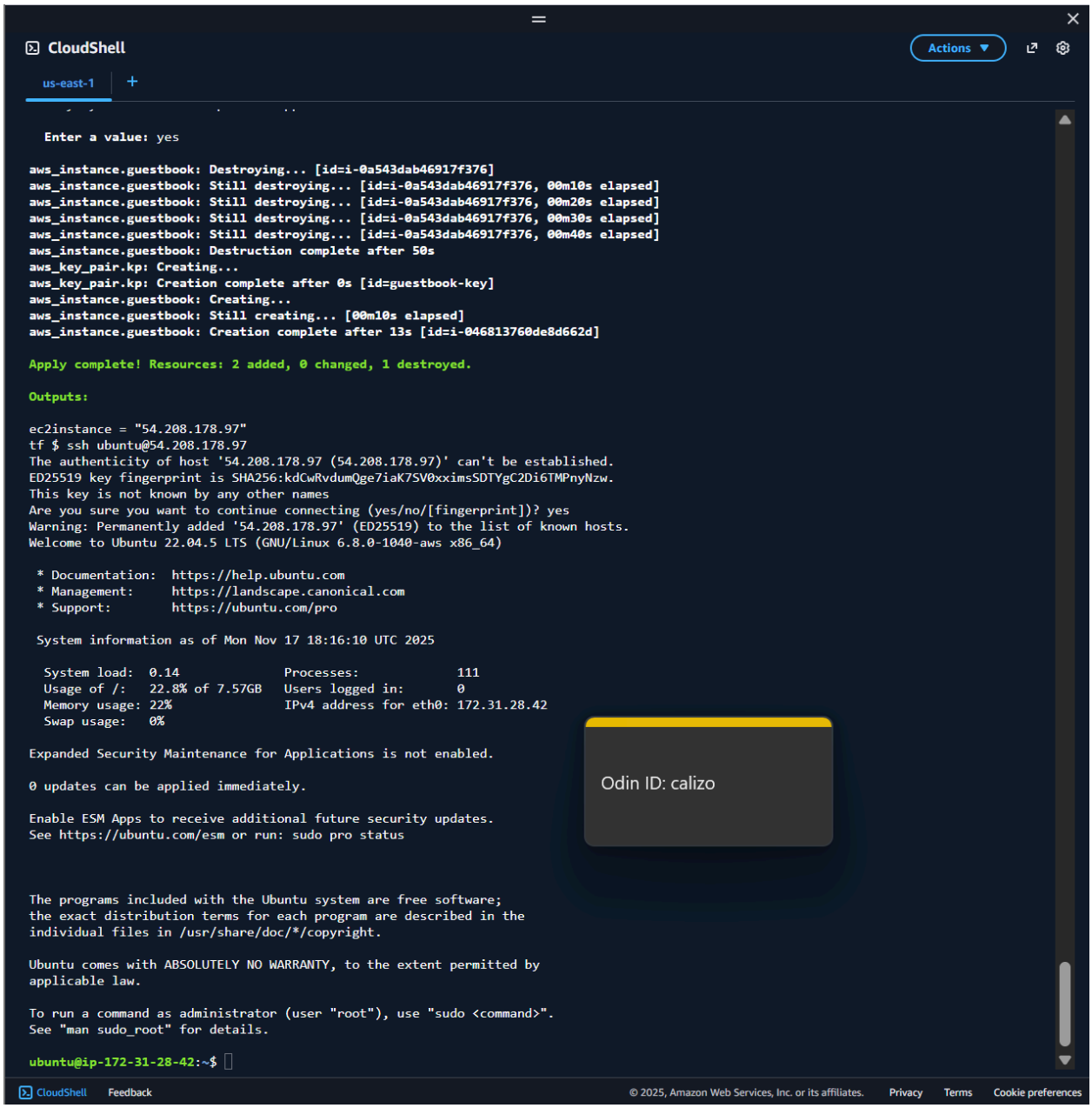
After attaching the security group to the instance, I ran another terraform plan and terraform apply. The screenshot confirms Terraform recognized the updates cleanly and reconfigured the instance without recreation. Watching the changes propagate reinforced the beauty of declarative infrastructure: one edit to the file, and the world conforms to it.

Step 6. Adding ssh access

To test connectivity, I attempted to SSH into the VM. As expected, the login failed because the VM had no authorized keys yet. I generated a new Ed25519 keypair in CloudShell and added an aws_key_pair block to main.tf, wiring its public key into the instance by referencing key_name in the EC2 resource.

After another apply, the instance accepted my SSH attempt instantly. The screenshot shows the moment of that successful login, the Ubuntu prompt blooming into view like a confirmation that Terraform had woven everything together exactly as intended.

1. Take a screenshot of the successful ssh login from Cloud Shell.



This shows after connecting to the instance over port 22 following the key-pair addition. This moment verifies that Terraform successfully bound my CloudShell public key to the ubuntu account.

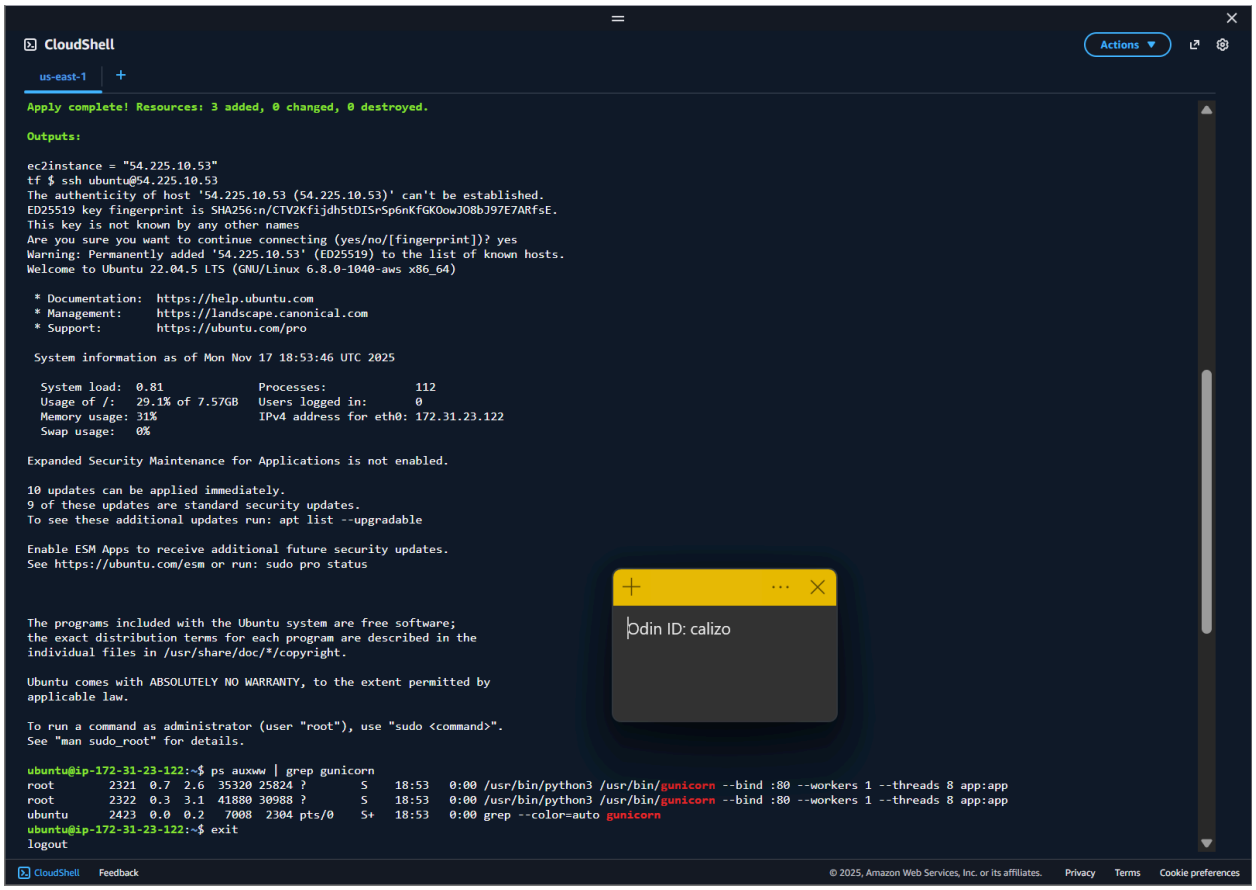
Step 7. Adding the Guestbook application

To transform this VM from a bare Ubuntu host into a functional application server, I created an install.sh file containing package installations, a git clone of the course repository, and a final gunicorn command bound to port 80. Including this script as user_data meant the VM would self-configure the moment it came alive.

A fresh terraform apply spun up a new instance, and the screenshot validates this deployment by showing the output block containing the new public IP address.

Once the machine finished bootstrapping, I SSHed in and checked for the gunicorn process. The screenshot shows its presence, confirming the Guestbook backend was awake and serving.

1. Take a screenshot of the output of the command that includes the IP address of the instance



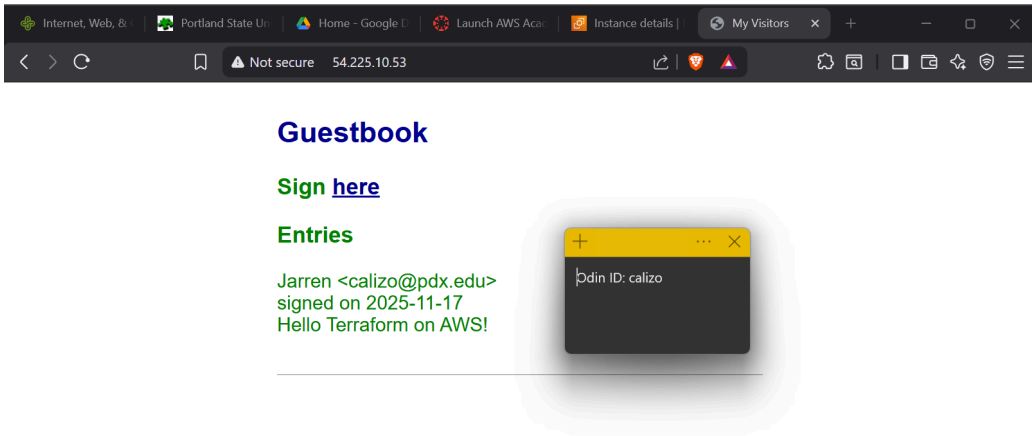
This shows after the apply completes, showing the ec2instance output containing the instance's public IP address.

Step 8. View the Guestbook

With the infrastructure assembled and the application running, I visited the VM's public IP address in my browser. The Guestbook loaded successfully. I signed it using my name, PSU e-mail, and the message "Hello Terraform on AWS!", watching my entry appear at the top of the page.

The screenshot captures this full moment: the URL bar pointing directly to the EC2 instance, and my message displayed within the Guestbook entries. Even though the page is simple, it represents a full chain of IAC, bootstrapping, and provisioning working in concert.

1. Take a screenshot of the Guestbook including the URL with the entry in it.



This shows after navigating to the instance's external IP and adding the required message. The URL and the entry are visible together.

Step 9. Clean Up

Once everything had been demonstrated successfully, I brought the deployment down with terraform destroy. Watching Terraform unwind the resources it had woven earlier in the lab reinforced the lifecycle completeness of IAC: creation, operation, validation, and teardown, all controlled from the same source file. After destruction, I optionally removed the temporary /tmp/tf directory to return the environment to a clean slate.

Lab 7.1g Terraform GCP Guestbook

Step 1. Terraform

This lab begins with the same principle that underpins all of Terraform: defining infrastructure not as a scattered set of console clicks, but as a single, declarative truth that can be versioned, replayed, and trusted. This step served mainly as conceptual grounding before any real configuration work began.

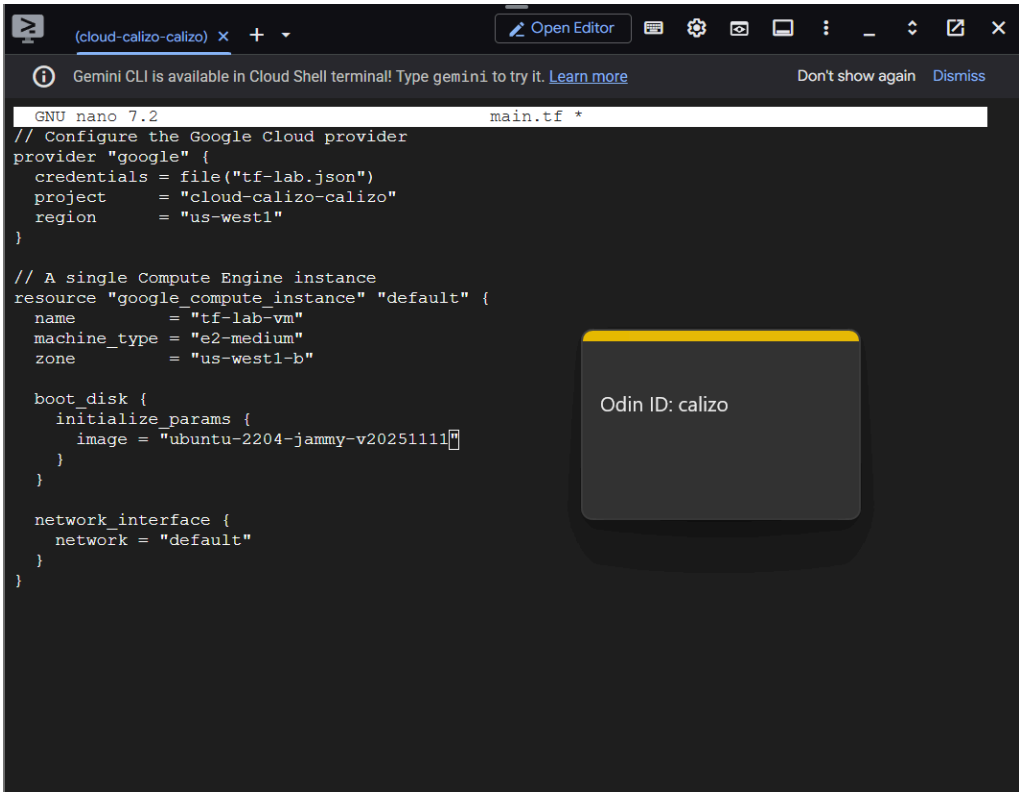
Step 2. Setup

After opening Cloud Shell, I verified Terraform's availability with a quick version check. With that in place, I created a working directory named tf and prepared Terraform to operate with the correct project permissions.

The service account creation, IAM binding, and key generation formed the backbone of Terraform's identity for the remainder of the lab. Each CLI command executed cleanly, and the tf-lab.json key was placed into the working directory as required. This setup framed the rest of the lab with a clear separation of concerns: Cloud Shell would orchestrate the commands, while Terraform would act through the service account identity I configured here.

Step 3. Initial Configuration

With the environment ready, I created main.tf and added the google provider block. Your first screenshot reflects this moment, showing the provider configured with tf-lab.json, the correct project ID, and region us-west1.

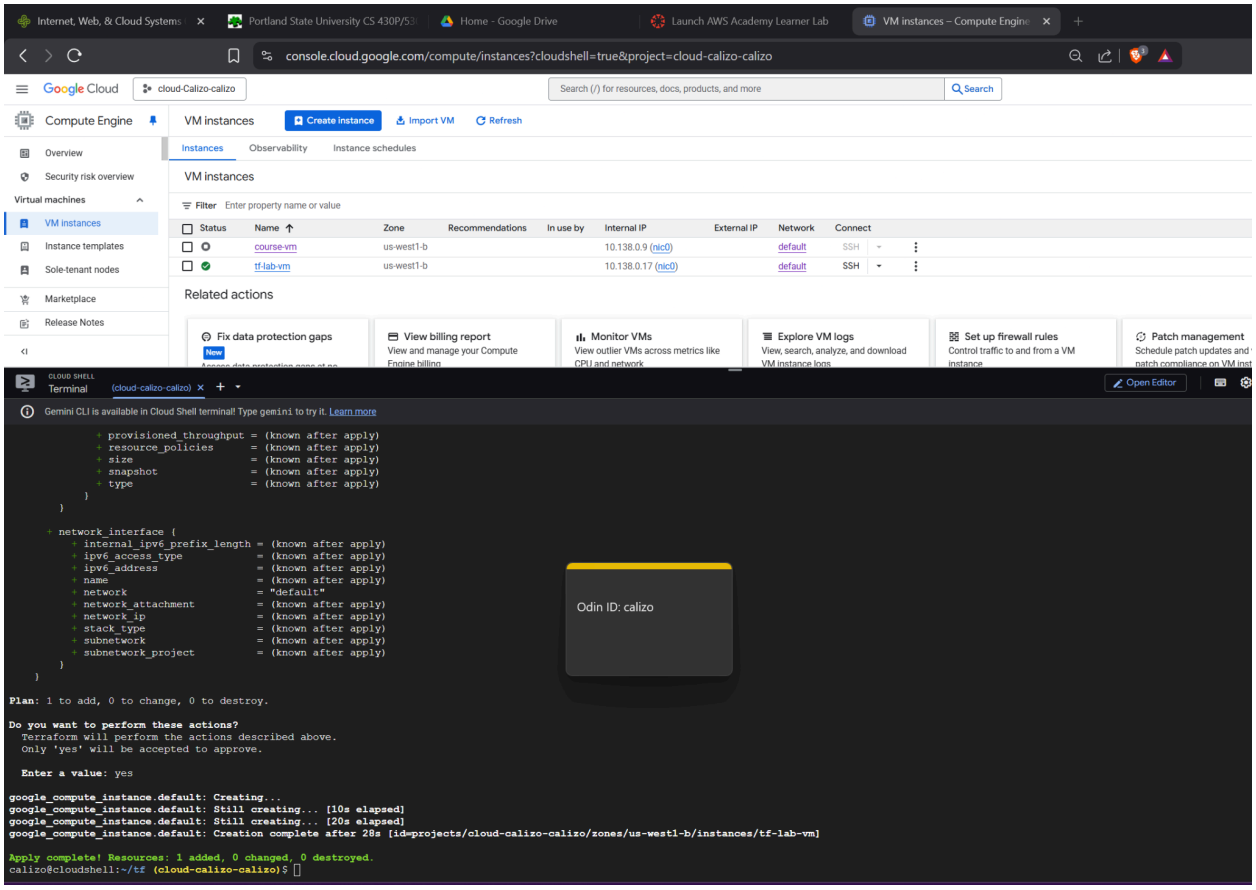


To choose the VM image, I listed Google's standard Ubuntu images and selected the most recent amd64 variant of Ubuntu 22.04. That image was then inserted into the google_compute_instance resource block, along with the e2-medium machine type and zone us-west1-b. The second screenshot confirms the first successful apply, showing the VM created without external IP access.

Step 4. Launching Configuration

I initialized Terraform using terraform init, then ran terraform plan to preview the creation of a single Compute Engine VM. As shown in the screenshot, terraform apply executed smoothly and the VM appeared in the Compute Engine console.

1. Take a screenshot that includes the VM's IP addresses



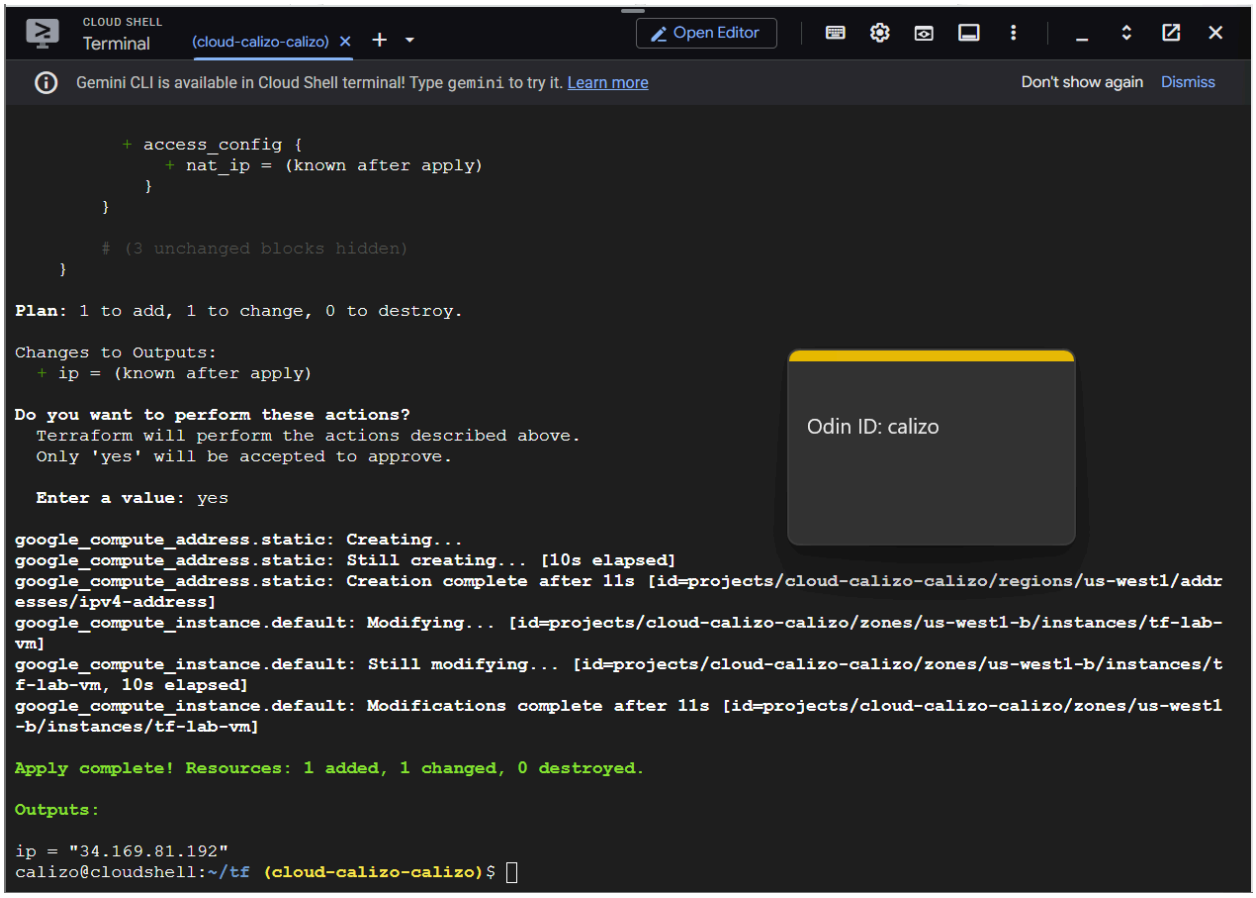
This shows the instance tf-lab-vm with only an internal IP assigned. This confirms Google's default networking, which does not automatically allocate an external IP.

Step 5. Adding an external IP address

To expose the VM to the outside world, I added a google_compute_address resource before the VM declaration. Then, I modified the instance's network_interface block to attach that static IP via an access_config stanza. An output block was added to surface the new external IP after apply.

Running terraform plan correctly showed one resource added (the static IP), one changed (the VM), and a new output exposed. The next screenshot captures terraform apply completing with all expected results, including the newly assigned static IP.

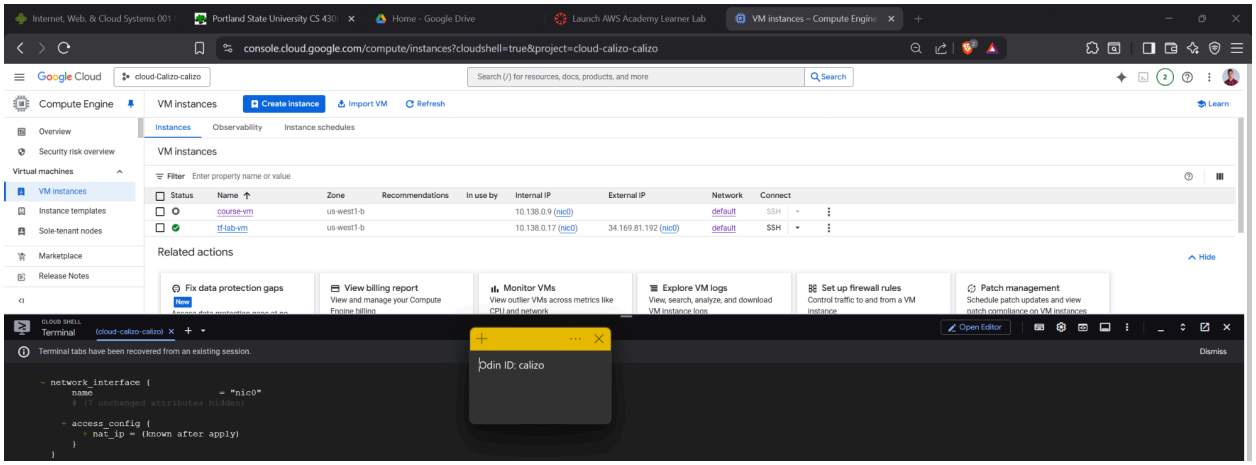
1. Take a screenshot showing the completion of the command including its output



This is showing the apply completion includes the new output "ip", verifying the address was created and attached to the VM.

Next, refreshing the VM instances view in the console confirmed both the internal and external IP addresses were now bound to tf-lab-vm. The final screenshot displays this clearly.

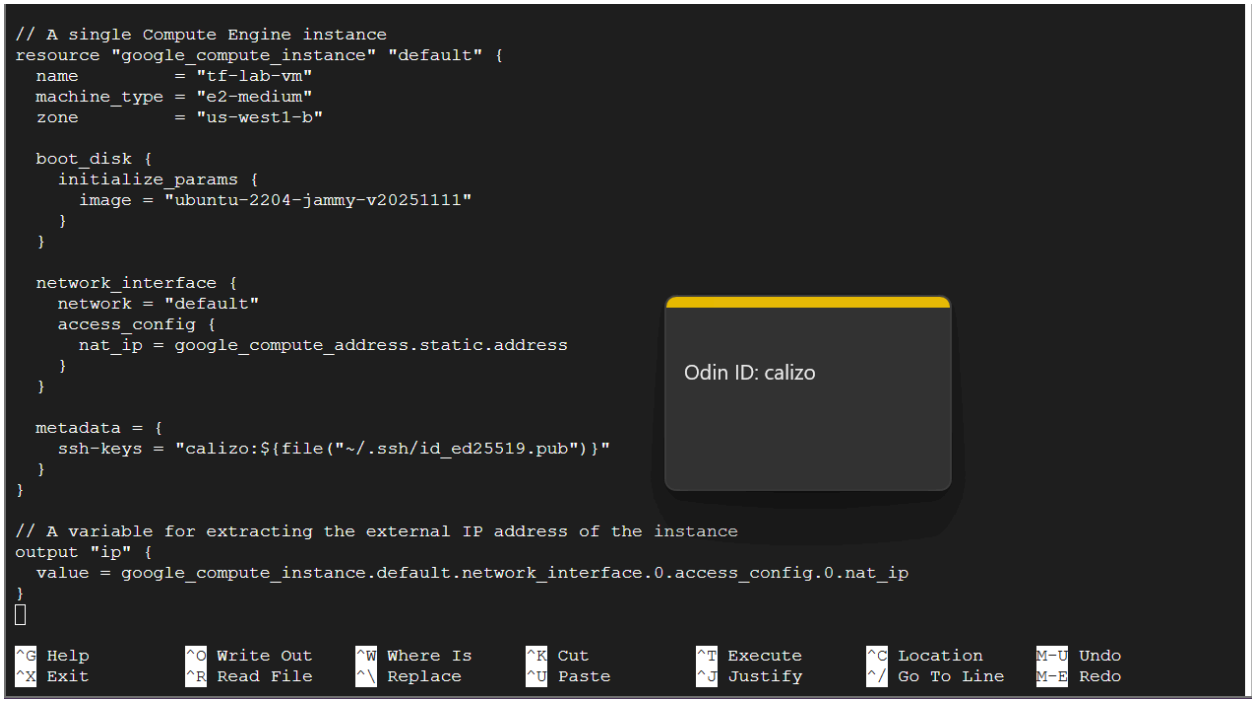
2. Take a screenshot that includes the VM's IP addresses



This shows tf-lab-vm with an internal address and the newly attached external IPv4 address, verifying the successful update to the VM's network configuration.

Step 6. Adding ssh access

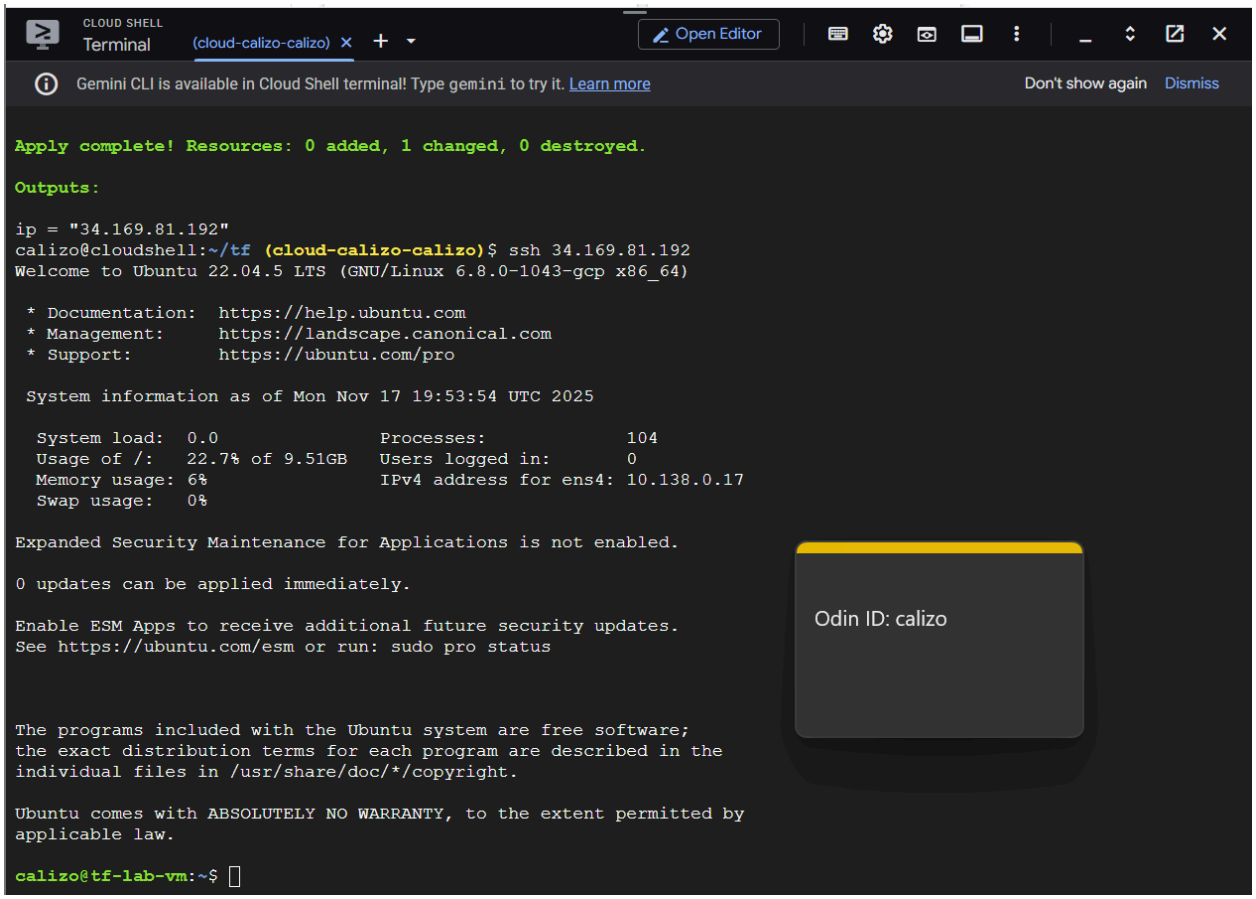
With the VM now publicly reachable, I attempted to SSH into its external IP. As expected, GCP rejected the connection because no authorized keys existed for the instance. I generated a new ed25519 keypair in Cloud Shell and updated the google_compute_instance block to include the public key in the metadata field.



After running terraform plan and terraform apply, Terraform recreated the metadata and applied the new SSH setting. The screenshot confirms the successful login: the familiar Ubuntu system banner greeted me with my username attached to the VM.

This moment always feels like stepping across the boundary between infrastructure and the living, breathing environment it creates.

1. Take a screenshot of the successful ssh login from Cloud Shell.



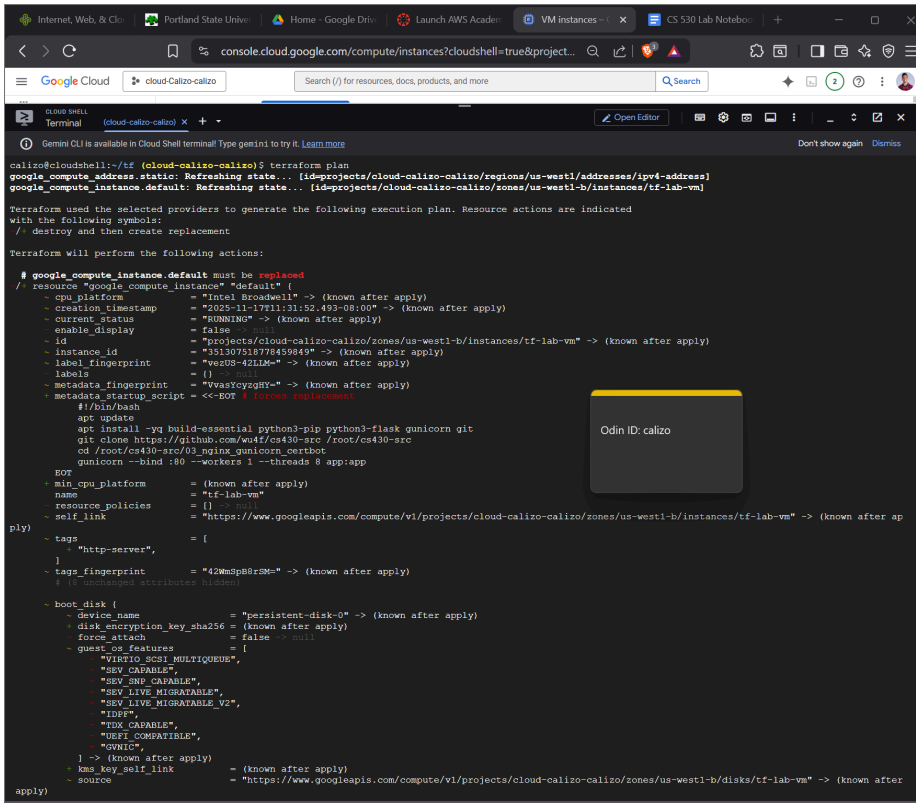
This shows a complete SSH login into the VM using the new ed25519 key, confirming that the metadata injection worked as intended.

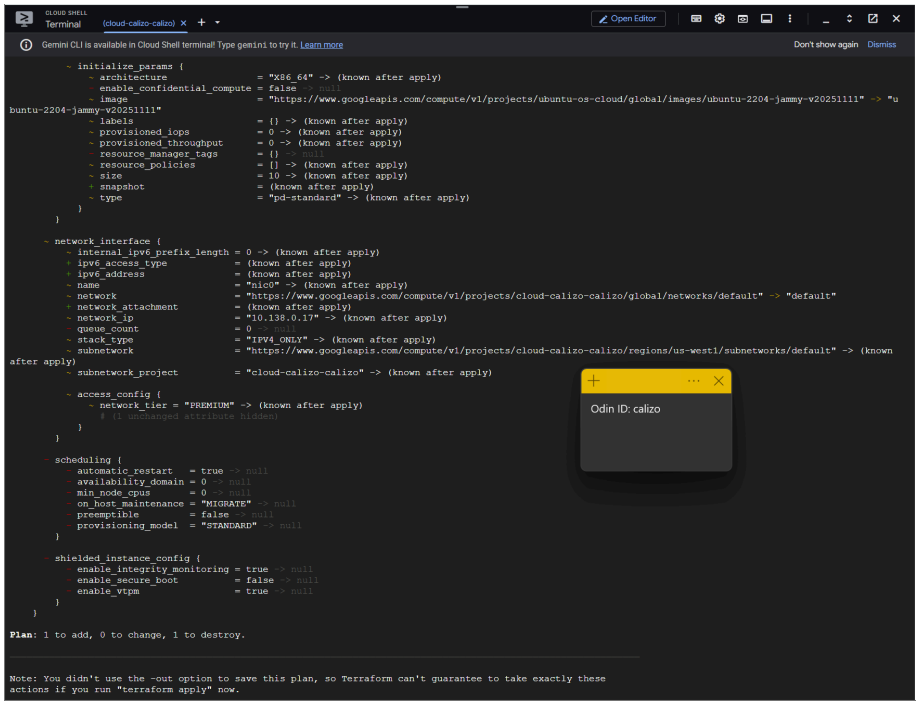
Step 7. Adding the Guestbook application

To turn this VM into a functioning web host, I created an install.sh startup script containing package installs, the cs430-src repo clone, and a gunicorn command bound to port 80. I then added metadata_startup_script into the VM resource so GCP would execute the script on boot.

Next, I added the "http-server" tag to the instance so GCP's predefined firewall rule would allow inbound port 80 traffic.

Running terraform plan reflected a deeper truth of Terraform: even a small configuration change can ripple into a resource replacement. In this case, adding the startup script and tag modified fields that require a recreate operation for the instance.



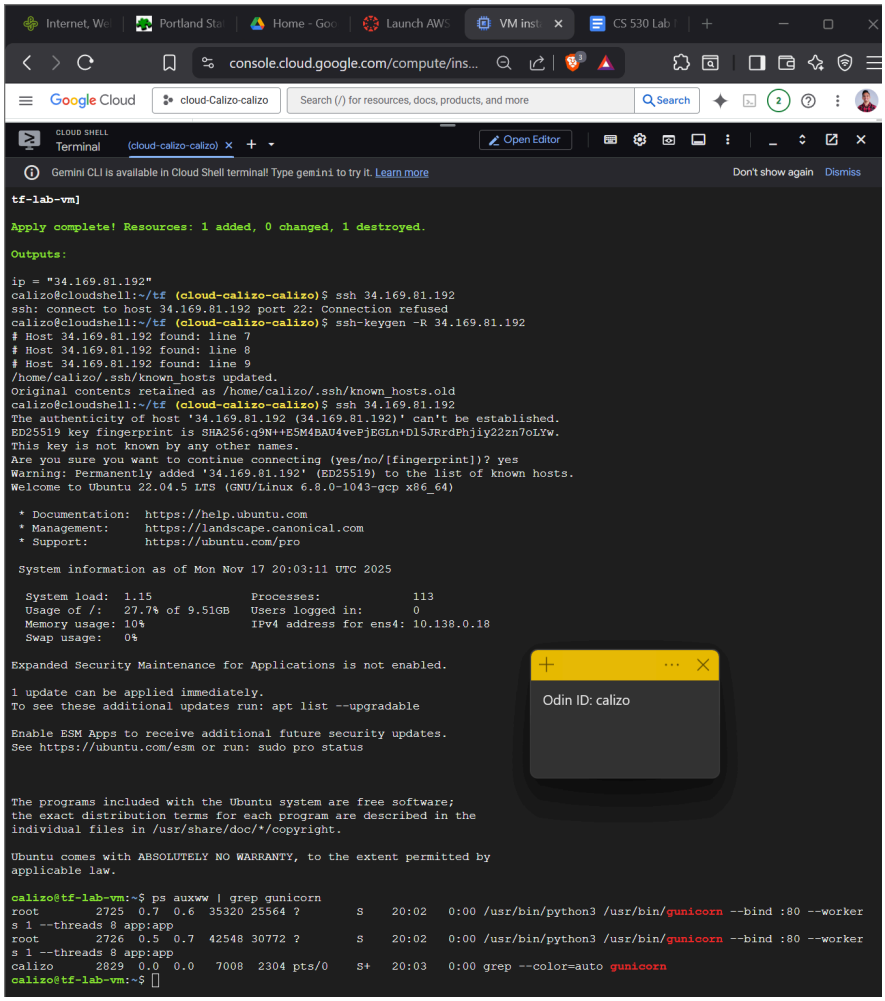


1. What resources are being added, changed, or destroyed?

Terraform showed that one resource would be added (the static IP was already created earlier), zero resources would be changed, and one resource (the VM) would be destroyed and replaced.

2. What part of the configuration forces a replacement to occur?

The replacement was triggered by adding metadata_startup_script and tags. Both fields fundamentally alter how the instance initializes and therefore force Terraform to recreate it from scratch.

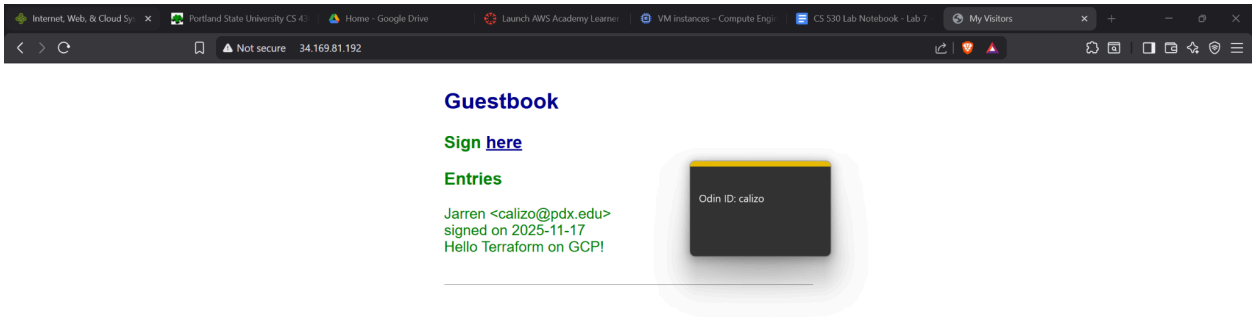


After apply, I SSHed back into the VM. Running ps auxww | grep gunicorn confirmed that gunicorn was alive and serving on port 80.

Step 8. View the Guestbook

With gunicorn running, I opened the external IP in my browser. The Guestbook loaded instantly. I left my entry using my name, email, and the message "Hello Terraform on GCP!". The final screenshot shows the live page, with my entry reflected back through the simple, quiet beauty of a working deployment.

1. Take a screenshot of the Guestbook including the URL with the entry in it.



This displays the Guestbook in the browser, including the URL and the entry I added.

Step 9. Clean Up

After confirming everything worked, I cleaned up the deployment. terraform destroy removed all created compute resources. I retrieved the KEY_ID from tf-lab.json, deleted the service account key, removed the Compute Admin policy binding, and finally deleted the service account itself. The directory cleanup closed the loop on the lab.

Even in the teardown, Terraform reminds you of the elegance of infrastructure as code: everything you create, you can also uncreate with equal precision.

Lab 7.2g Kubernetes Guestbook

Stepping into Kubernetes felt like stepping into a wider horizon of orchestration, where containers no longer live as isolated units but breathe together through pods, replicas, and nodes. The Guestbook that once ran as a VM and then as a Cloud Run service is now being shaped into a distributed application managed by a cluster that watches itself, heals itself, and keeps the promises we declare in YAML. This lab guided me through that transition, piece by piece.

Step 1. Kubernetes Overview

Before touching any commands, I revisited the fundamental mental model Kubernetes expects: nodes are the machines, pods are the units of execution, and replicas are how we scale. The lab image showed a clean visual of how 3 replicas sit across 2 worker nodes. The idea is simple but powerful. Kubernetes is essentially building a serverless platform out of raw compute, and I get to steer it.

Step 2. Setup

From Cloud Shell, I enabled the APIs needed for GKE and Compute Engine. It felt like opening the gatekeepers for orchestration.

- gcloud services enable container.googleapis.com
- gcloud services enable compute.googleapis.com

After listing zones, I anchored the environment to us-west1-b:

- gcloud config set compute/zone us-west1-b
- gcloud config list

Step 3. Assigning Privileges

Each node in the cluster must be able to read and write to Cloud Datastore and pull images from Artifact Registry. Instead of creating a fresh service account, I augmented the existing Guestbook service account with the Artifact Registry reader role:

```
gcloud projects add-iam-policy-binding cloud-calizo-calizo \
  --member serviceAccount:guestbook@cloud-calizo-calizo.iam.gserviceaccount.com \
  --role="roles/artifactregistry.reader"
```

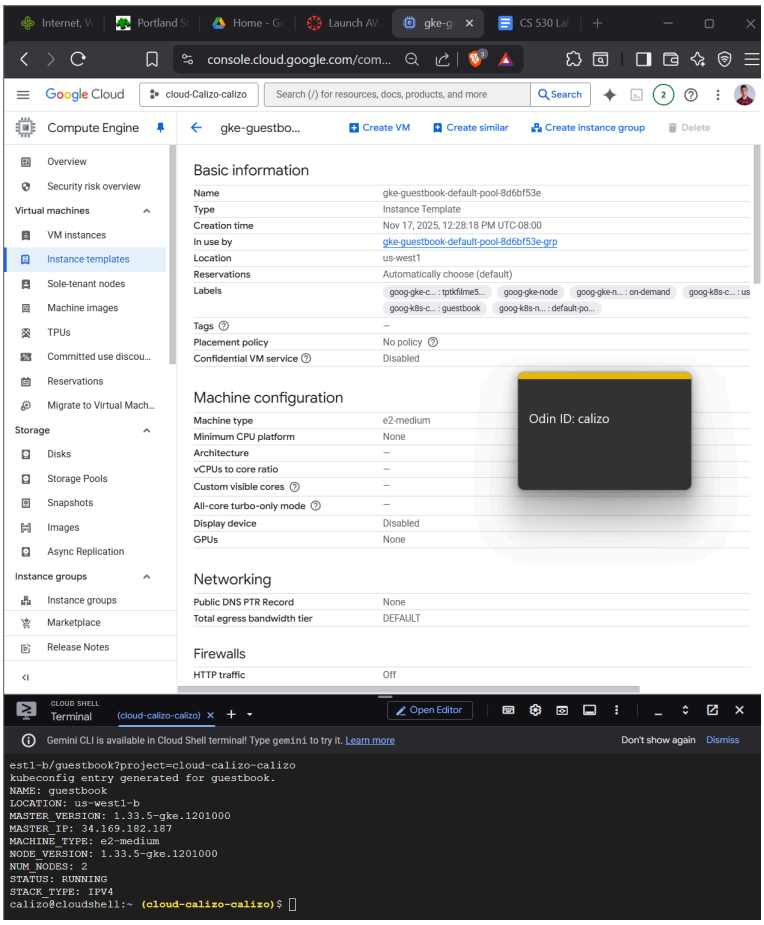
This ensures the nodes will be able to pull the container image once deployed through Kubernetes.

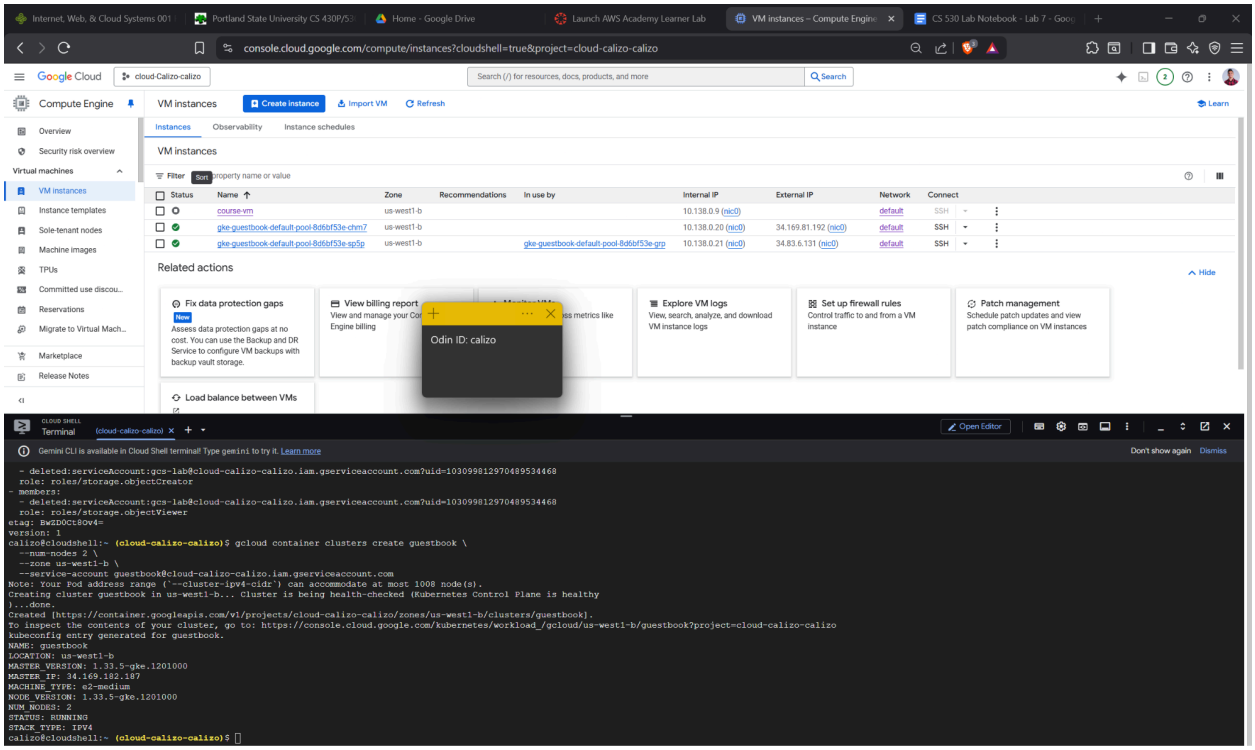
Step 4. Create Kubernetes cluster

With permissions set, I spun up the cluster:

```
gcloud container clusters create guestbook \
  --num-nodes 2 \
  --zone us-west1-b \
  --service-account guestbook@cloud-calizo-calizo.iam-gserviceaccount.com
```

The cluster came online cleanly. From Compute Engine, I navigated to the instance templates and instance groups auto-generated by GKE.





Based on the screenshots:

1. What is the name of the Instance Template dynamically generated to create the two nodes (VMs)?

- gke-guestbook-default-pool-8d6bf53e

2. What is the name of the Instance Group dynamically generated that the two nodes belong to?

- gke-guestbook-default-pool-8d6bf53e-grp

3. What are the names of the two nodes?

From the VM instances list, the two nodes were:

- gke-guestbook-default-pool-8d6bf53e-chm7
- gke-guestbook-default-pool-8d6bf53e-ss9b

These represent the worker nodes where pods will eventually land.

I then set kubectl's default cluster context:

- gcloud config set container/cluster guestbook

Step 5. Prepare a container image

I returned to the Cloud Datastore Guestbook source:

- cd cs430-src/05_gcp_datastore

Then authenticated Docker for Artifact Registry:

- gcloud auth configure-docker

Built and tagged the image:

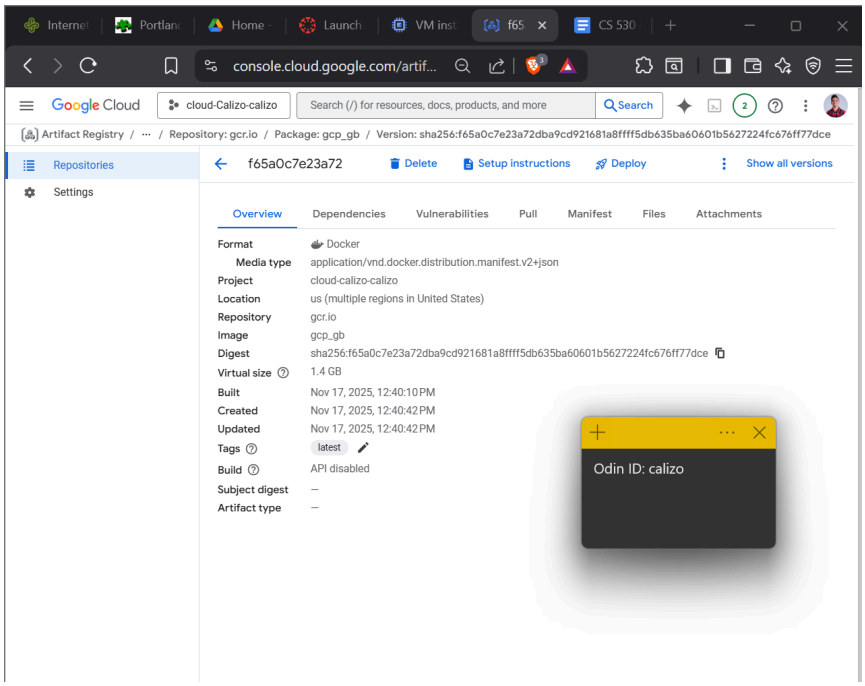
- docker build -t gcr.io/cloud-calizo-calizo/gcp_gb .

Pushed it:

- docker push gcr.io/cloud-calizo-calizo/gcp_gb

Navigating to Artifact Registry showed the container image successfully created and tagged latest.

1. Take a screenshot of the container image created



Step 6. kubernetes.yaml

In kubernetes.yaml, I filled in the project ID inside the image field:

- image: gcr.io/cloud-calizo-calizo/gcp_gb

Everything else matched the lab spec: 3 replicas, containerPort 8000, and a LoadBalancer service mapping port 80 to targetPort 8000.

Step 7. Deploy the configuration

First, I synchronized local kubectl credentials with the new cluster:

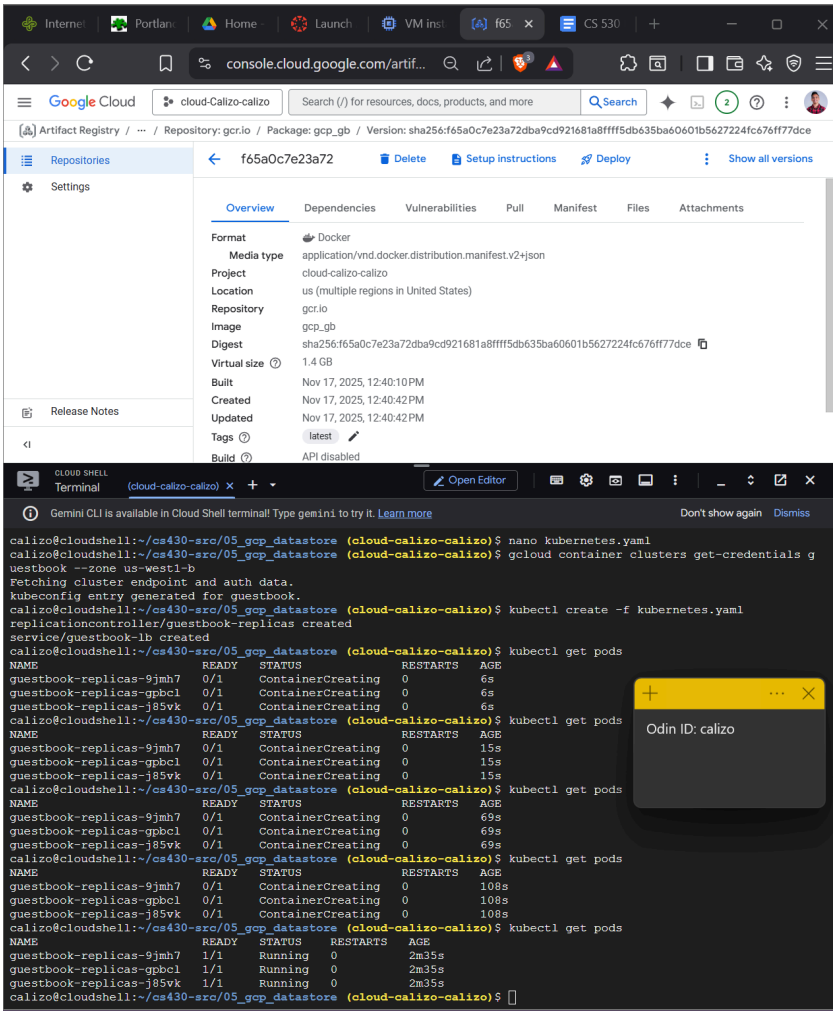
- gcloud container clusters get-credentials guestbook --zone us-west1-b

Then deployed the configuration:

- kubectl create -f kubernetes.yaml

The pods went through the ContainerCreating state and settled into Running across all three replicas.

1. Take a screenshot of the output of the following command when all 3 replicas reach a "Running" state.

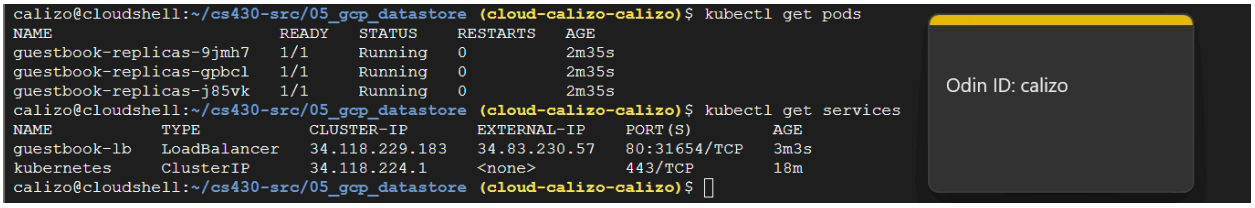


Then I inspected the service:

- kubectl get services

A LoadBalancer external IP appeared, validating that the application is now reachable.

2. Take a screenshot of listing services with LoadBalancer indicating an external IP address that is ready for access.



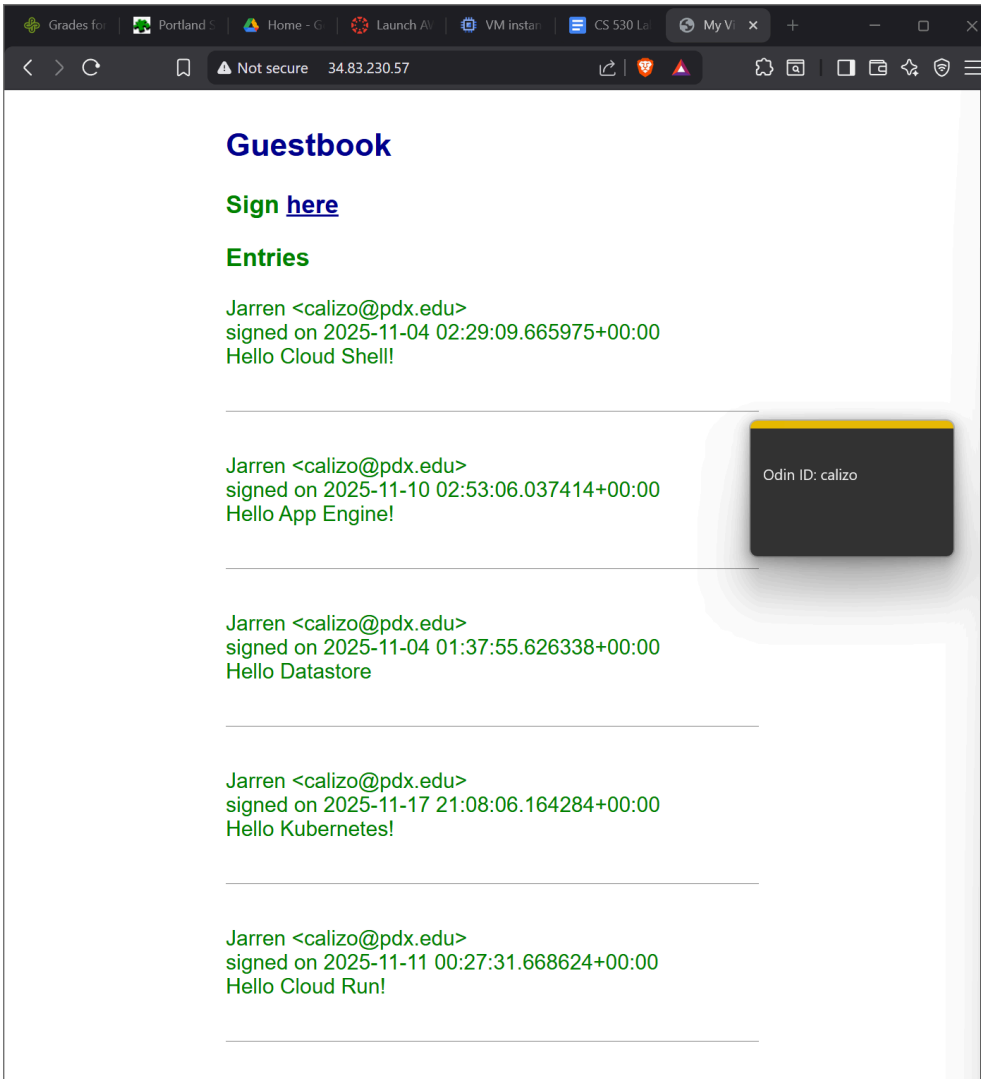
Insight

These steps brought together everything learned across the course so far. With Kubernetes, the Guestbook no longer feels like a single machine artifact. It becomes something that lives across nodes, scales through replicas, and is exposed via a clean LoadBalancer entry point. The YAML declaration quietly governs the cluster, and GKE keeps the promises alive.

Step 8. View the Guestbook

The moment the LoadBalancer’s external IP came alive in the browser felt like a quiet crescendo in this long Kubernetes symphony. I typed in my entry, let the request disappear into the unknown depths of the cluster, and received it back on the screen as if the cloud whispered it into existence. Stateless replicas, one Datastore backend, and a seamless layer of orchestration made the whole thing feel elegantly detached from the underlying complexity.

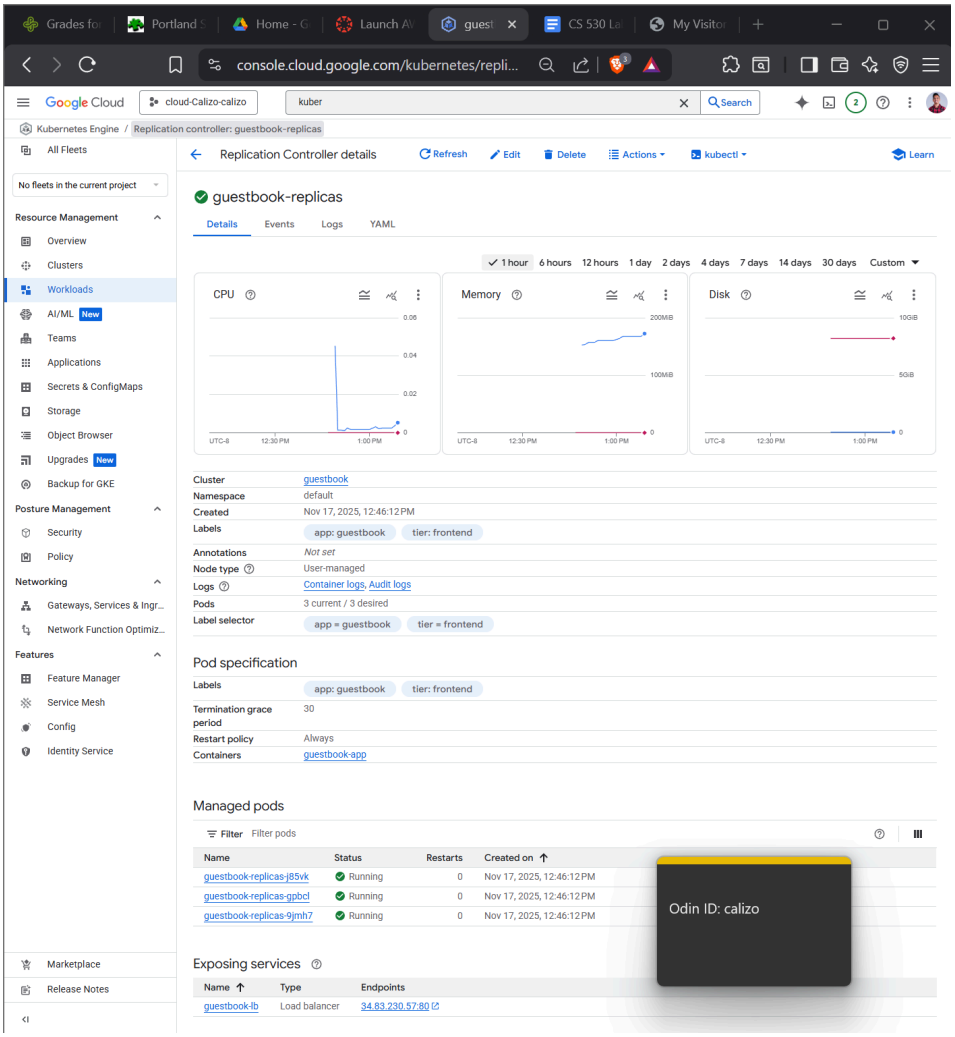
1. Take a screenshot of the Guestbook including the URL with the entry in it.



After submitting the entry, the Guestbook rendered it alongside previous versions from Cloud Shell, App Engine, Datastore, Cloud Run, and the earlier Kubernetes deployment. This validated the end-to-end pipeline from container to replica to Datastore.

Stepping into Kubernetes Engine's Workloads view felt like peeling back the curtain on a well-rehearsed performance. The three pods under the guestbook-replicas controller were healthy, synchronized, and reporting metrics without a hiccup.

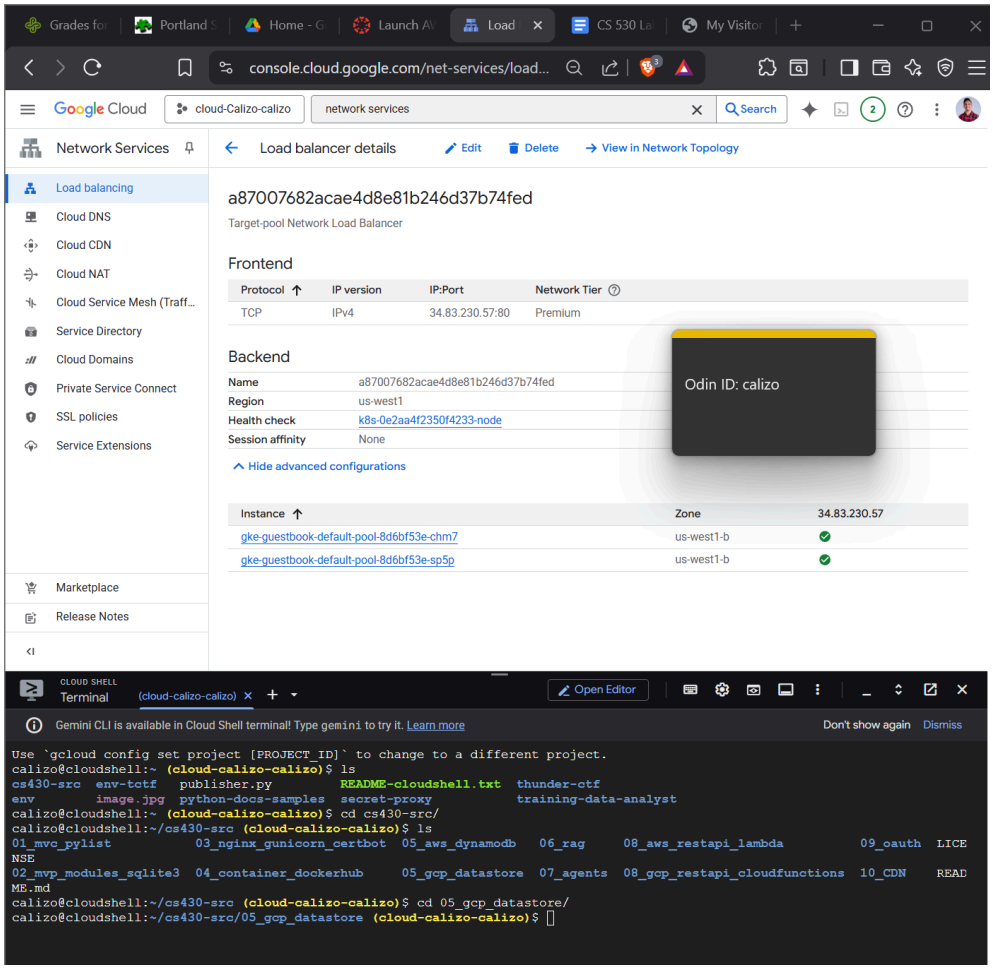
2. Take a screenshot of the managed guestbook pods and the service being exposed.



I could see all three replicas running, each with the expected labels, restart counts at zero, and the service guestbook-lb clearly linked as the frontend exposure point.

Next, I traced the load-balancing layer that sat between users and the replicas. The Cloud Load Balancer mapped port 80 on the external interface back to port 8000 on the containers, with health checks targeting each node pool VM. It felt like watching cloud infrastructure breathe: balancing requests, verifying health, and quietly routing traffic without complaint.

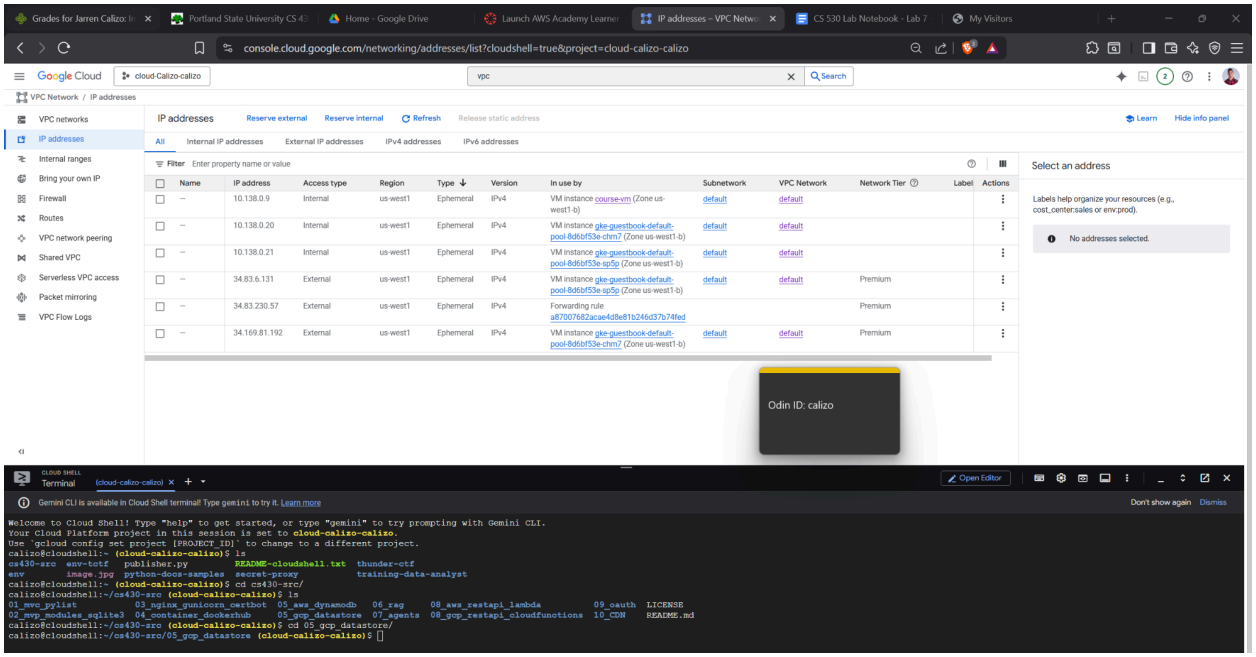
3. Take a screenshot of the load balancer and its details



The load balancer showed the external IP, backend instances, health checks, and forwarding rule.

Finally, I catalogued the IP addresses that Kubernetes allocated. Two belonged to the nodes themselves, while a separate one was reserved for the load balancer. Seeing them all together in the VPC UI gave me a full map of how Kubernetes carved out networking for the cluster.

4. Take a screenshot of the addresses allocated and indicate the ones associated with nodes versus the one associated with the load balancer.



- Node IPs: 34.83.6.131 and 34.83.6.131 (based on the two gke-guestbook instances)
- Load Balancer IP: 34.83.230.57
- The previously used VM external IP: 34.169.81.192 (unrelated to this cluster, but still present)

Step 9. Delete workload and service

To bring down the deployment, I executed:

- kubectl delete -f kubernetes.yaml

This removed the ReplicationController and the LoadBalancer service.
Then I deleted the container image:

- gcloud container images delete gcr.io/cloud-calizo-calizo/gcp_gb

The Kubernetes cluster itself remained running, waiting for the CI/CD automation stage.

Step 10. CI/CD Build Automation

Before delegating deployments to Cloud Build, I granted its service account the container.developer role so it could manage resources on the cluster. Capturing the project number and binding the policy brought Cloud Build into the loop:

```
GOOGLE_CLOUD_PROJECT_NUMBER=$(gcloud projects describe cloud-calizo-calizo
--format="value(projectNumber)")
gcloud projects add-iam-policy-binding cloud-calizo-calizo \
  --member
serviceAccount:${GOOGLE_CLOUD_PROJECT_NUMBER}@cloudbuild.gserviceaccount.co
m \
  --role=roles/container.developer
```

This permission step ensures Cloud Build can not only build images but also deploy them using gke-deploy.

Step 11. Configure build automation

Inside the Datastore application directory, I authored cloudbuild.yaml to orchestrate a full build-and-deploy flow:

- Build Docker image
- Push to Artifact Registry
- Deploy to Kubernetes via gke-deploy

It acts like a Makefile for the cloud, chaining the entire delivery pipeline into one command.

Step 12. Deploy and view application

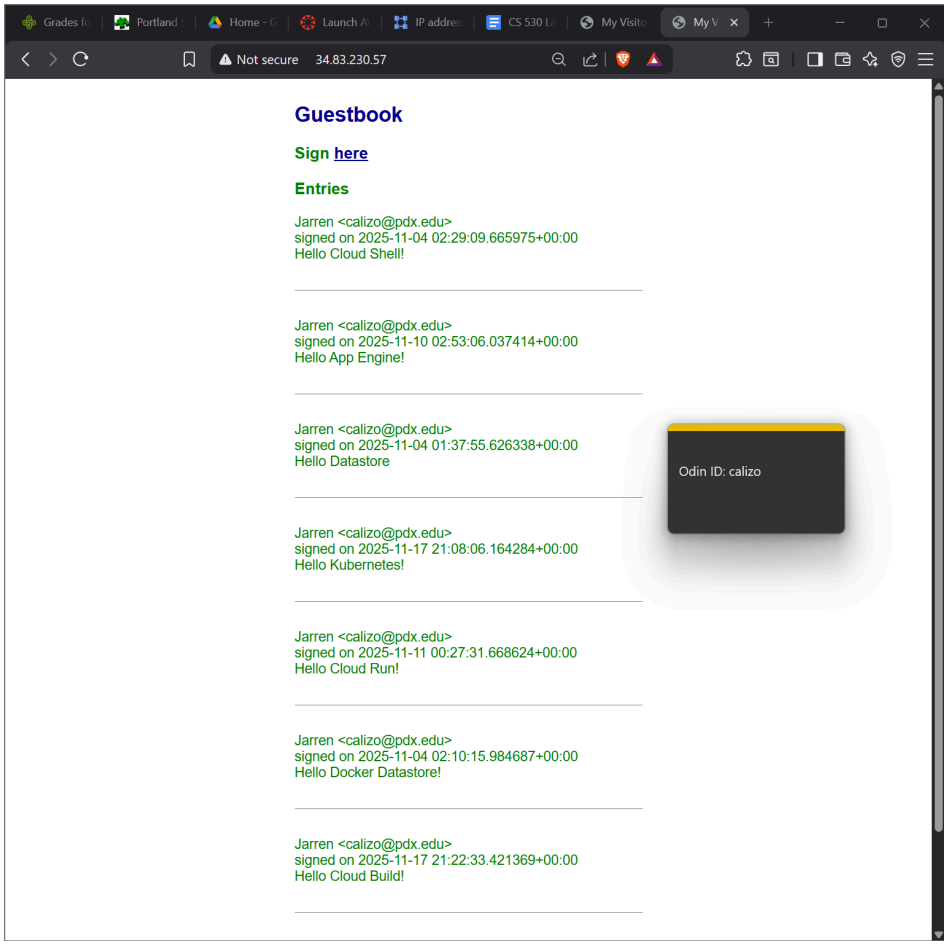
I triggered the whole automation pipeline with:

- gcloud builds submit --config=cloudbuild.yaml

Cloud Build rebuilt the image, pushed it, and redeployed the workload and service onto the Kubernetes cluster automatically.

After waiting briefly for replicas to spin up, I revisited the load balancer’s IP to confirm that the newly deployed containers were active. Then I signed the Guestbook again with: "Hello Cloud Build!"

1. Take a screenshot of the Guestbook including the URL with the entry in it.



This confirmed that CI/CD had correctly deployed the latest image and that Kubernetes seamlessly served the updated version.

Step 13. Clean up

Finally, I cleared out the deployment:

- kubectl delete -f kubernetes.yaml
- gcloud container images delete gcr.io/cloud-calizo-calizo/gcp_gb

Then, with one last command, I tore down the Kubernetes cluster:

- gcloud container clusters delete guestbook --zone us-west1-b

The cluster dissolved, and the once-living infrastructure returned to dust, ready for another cold start on another day.

Lab 7.3g APIs (Slack, Knowledge Graph)

There is something uniquely satisfying about watching a toolchain click into place. In this lab, I built a small bridge between Slack and Google's Knowledge Graph, letting a simple slash command summon structured intelligence on demand. The workflow was elegant: a Slack request signed with my workspace secret, a Cloud Function verifying the signature, a Knowledge Graph query executed through Google's discovery interface, and finally a formatted response posted back into Slack. It felt like wiring together two distant worlds and watching them speak smoothly for the first time.

Step 2. Code

Step 1. Slack and KG Setup

I began inside Cloud Shell by cloning Google's sample repository and navigating into the Slack example folder. From here, the scaffolding for kg_search awaited, showing how the Cloud Function ties Slack's request signature verification to the Knowledge Graph API call.

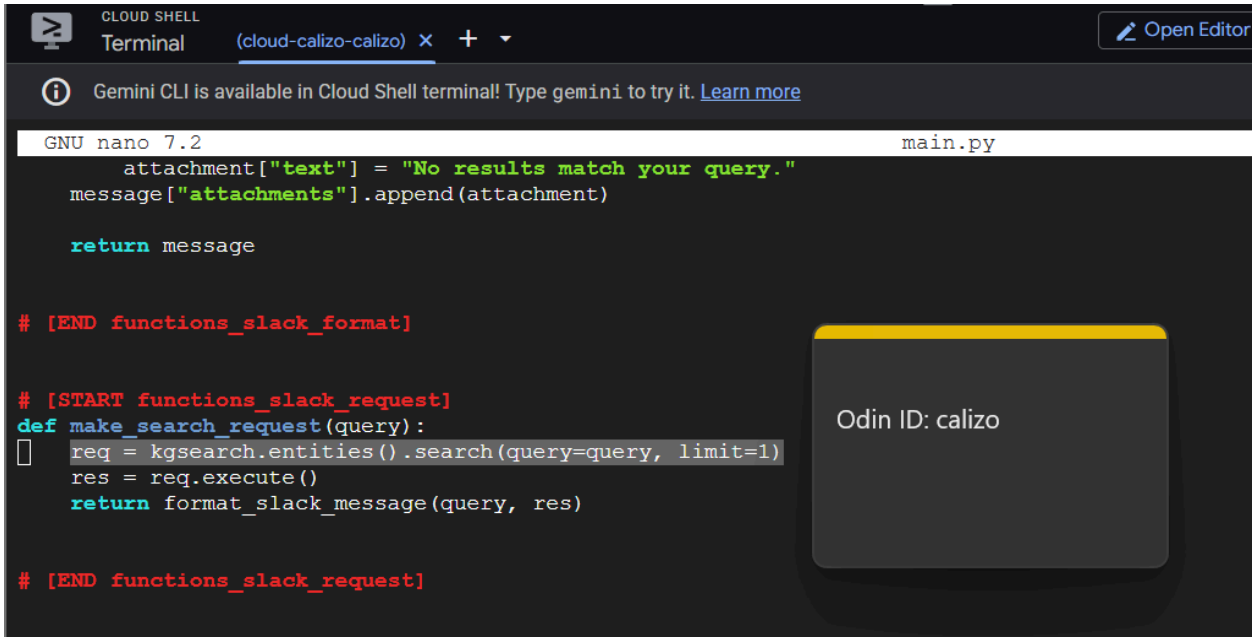
1. Does Google provide a Python package specifically for accessing the Knowledge Graph API?

- No. Google does not provide a dedicated Python client for the Knowledge Graph API. Instead, the lab uses the generic googleapiclient.discovery interface to instantiate an API object.

Step 3. Code

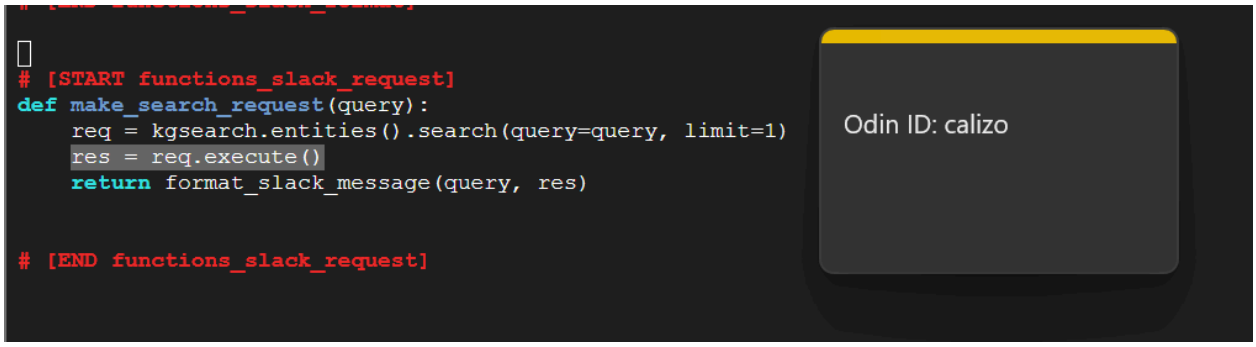
Inside main.py, I examined the section responsible for building the Knowledge Graph query. This portion reveals how the API discovery client creates a structured request object.

1. Show the source line that constructs the query we wish to send to the Knowledge Graph API.



req = kgsearch.entities().search(query=query, limit=1)

2. Show the source line that then executes the query and saves the response. What is the name of the method that sends the query to the Knowledge Graph API?



res = req.execute()

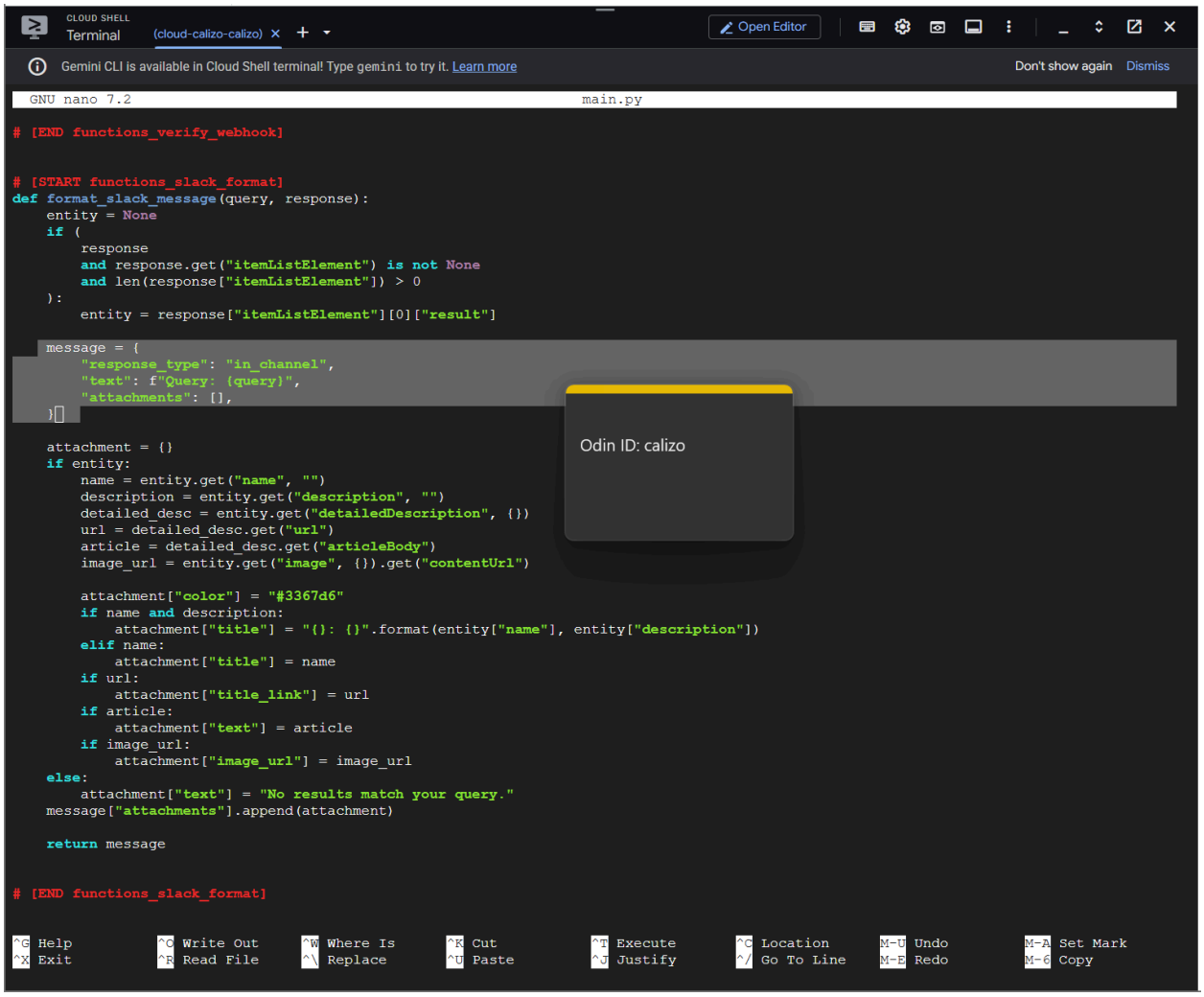
- The method that sends the query is the execute() method.

To shape the Knowledge Graph response into a Slack-friendly attachment, the code uses a clean structure within format_slack_message().

3. What is the Python data type that is used to represent the formatted message?

- A Python dictionary.

4. What are the three main attributes of the formatted message passed back to Slack?



The message includes:

- "response_type"
- "text"
- "attachments"

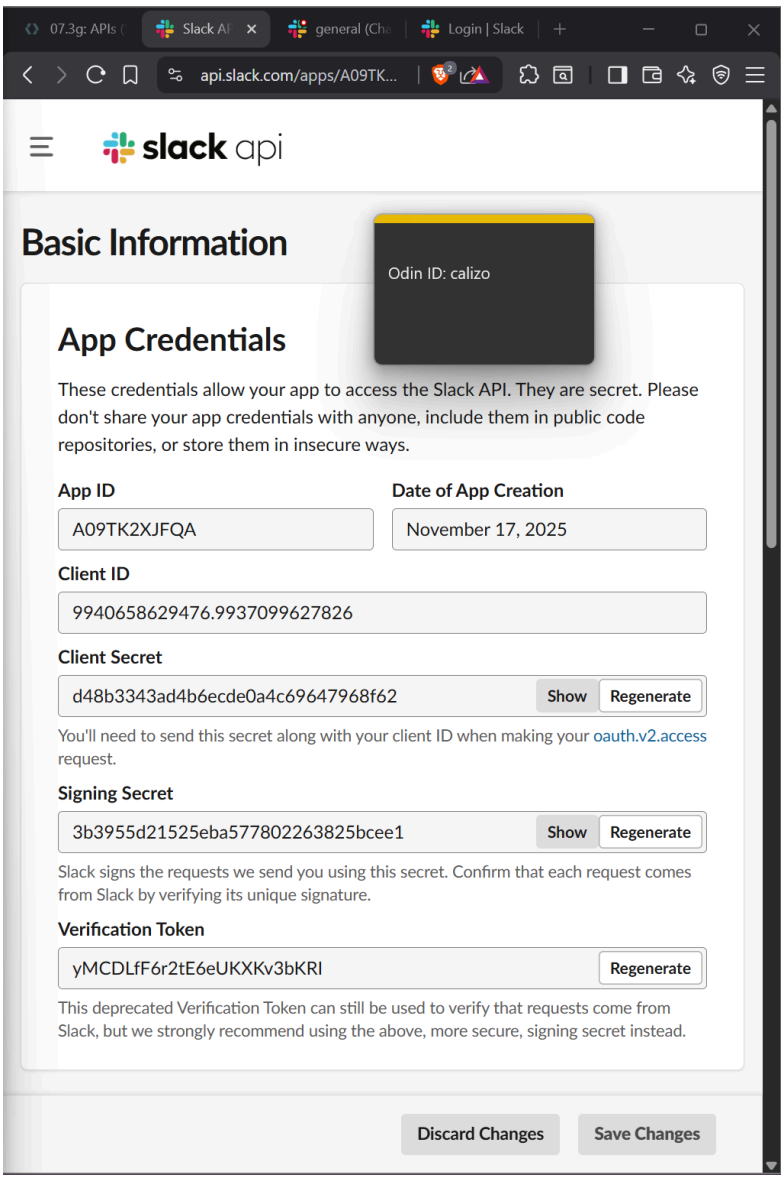
These form the backbone of the Slack response.

Step 4. Knowledge Graph API Configuration

I enabled the Knowledge Graph API and generated a restricted API key that would serve as the authentication layer between my Cloud Function and Google's KG backend. This key was later exported into the Cloud Function environment as KG_API_KEY.

Step 5. Slack Workspace and Signing Secret

I created a fresh Slack workspace and an associated Slack App. From the App Credentials page, I retrieved the Signing Secret. This is the shared truth that lets the Cloud Function verify Slack signatures through HMAC and reject any forged requests. This secret was later injected into the Cloud Function environment as SLACK_SECRET.



Step 6. Deploying the Cloud Function

Next, I deployed kg_search using Cloud Functions with both secrets set as environment variables:

```
gcloud functions deploy kg_search \
  --runtime python310 \
  --trigger-http \
  --set-env-vars "SLACK_SECRET=...,KG_API_KEY=..." \
  --allow-unauthenticated
```

The endpoint URL generated here became the address for Slack's slash command integration.

Step 7. Creating the Slack Command

Inside my Slack App settings, I created the /kg command and set its Request URL to the Cloud Function endpoint. With the app installed into my workspace, the command was now fully wired: Slack sends the request, the function validates it, the KG API returns the entity, and the formatted message flows back into the channel.

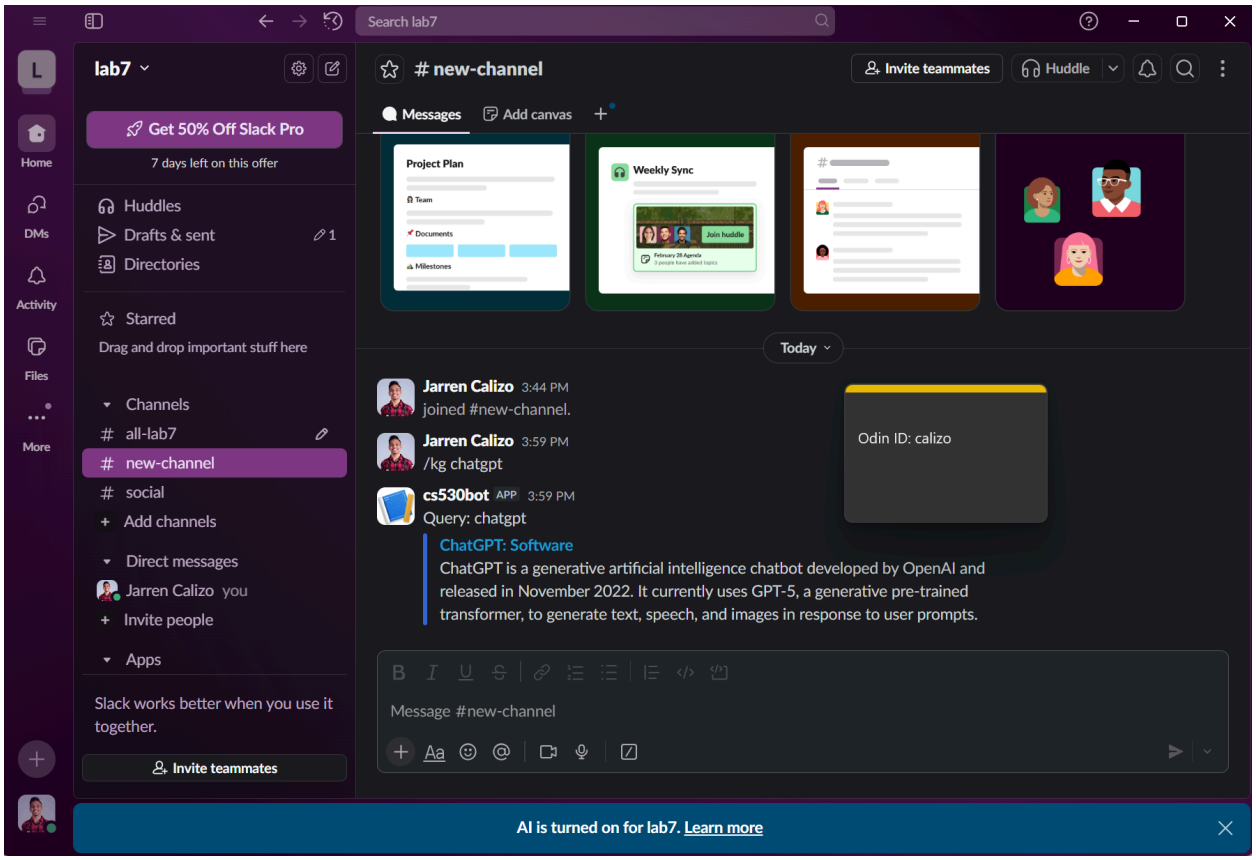
Step 8. Test the command

Inside my workspace's channel, I issued:

- /kg chatgpt

My Cloud Function responded exactly as intended, returning a Knowledge Graph result describing ChatGPT. The screenshot shows the full formatted block: the query echoed back, the title, and the Knowledge Graph description pulled directly from Google's dataset.

1. Take a screenshot of its response for your lab notebook.



This shows the /kg chatgpt command and the formatted Knowledge Graph response appearing correctly. This serves as proof that the Cloud Function endpoint, signature verification, Knowledge Graph API call, and Slack formatting were all working in harmony.

<u>Insight</u>

This lab was an exercise in gluing together authentication, APIs, and cloud automation into a tiny but expressive workflow. There was something almost poetic about watching Slack, Cloud Functions, and Google's Knowledge Graph hand off data across the stack, each layer trusting the next because the right secrets and signatures were aligned.

It was a reminder that even small integrations can feel like crafting a bridge between worlds, one clean interface at a time.

Lab 7.4 ML APIs

Stepping into this lab felt like opening a small toolbox of living intelligence. Vision, Speech, and Translate are APIs that carry more than functionality. They echo the feeling of standing at the edge of machine perception, watching code begin to understand what we see, hear, and say. My role here was to wire together service accounts, enable capabilities, and test each API through clean Python scripts. Each section became its own quiet victory, stitched together by concrete commands and real-time outputs inside Cloud Shell.

Step 1. Setting up the ML APIs

I began by enabling the four core APIs: Vision, Speech, Translate, and Natural Language. With Cloud Shell ready, I cloned the python-docs-samples repository, created my virtual environment with virtualenv -p python3 env, and activated it. This created a predictable, isolated environment for the rest of the lab.

Step 2. IAM Service Account Setup

To allow the ML APIs to accept authenticated calls, I created a dedicated service account named cs430mlapis. I granted it the roles/serviceusage.serviceUsageConsumer role, then generated its JSON key. After saving the key to cs430mlapis.json, I exported GOOGLE_APPLICATION_CREDENTIALS so each API call would quietly use the correct identity. This formed the trust boundary for all API usages in this lab.

Step 3. Vision

Label Detection

Using the Vision API examples, I changed into vision/snippets/detect and ran:

- python detect.py labels-uri gs://cloud-samples-data/ml-api-codelab/birds.jpg

1. Show the output for your lab notebook

```
(env) calizo@cloudshell:~/python-docs-samples/vision/snippets/detect (cloud-calizo-calizo)$ python detect.py labels-uri gs://cloud-sample
s-data/ml-api-codelab/birds.jpg
Labels:
Bird
Ratite
Common ostrich
Flightless bird
Beak
Vertebrate
Wildlife
Terrestrial animal
Greater rhea
Feather
(env) calizo@cloudshell:~/python-docs-samples/vision/snippets/detect (cloud-calizo-calizo)$
```

Odin ID: calizo

The screenshot shows the label predictions returned for the birds image: categories such as bird species, feathers, and descriptors tied to wildlife. This validated that Cloud Vision was operational.

Inside detect.py, I found the function handling this call.

```python
# [START vision_label_detection_gcs]
def detect_labels_uri(uri):
    """Detects labels in the file located in Google Cloud Storage or on the
    Web."""
    from google.cloud import vision

    client = vision.ImageAnnotatorClient()
    image = vision.Image()
    image.source.image_uri = uri

    response = client.label_detection(image=image)
    labels = response.label_annotations
    print("Labels:")

    for label in labels:
        print(label.description)

    if response.error.message:
        raise Exception(
            "{}\nFor more info on error messages, check: "
            "https://cloud.google.com/apis/design/errors".format(response.error.message)
        )

# [END vision_label_detection_gcs]
```

Odin ID: calizo

2. What is the name of the function?

- detect_labels_uri

3. What type of Vision client is instantiated in it?

- vision.ImageAnnotatorClient()

4. What method is invoked in the Vision client to perform the detection?

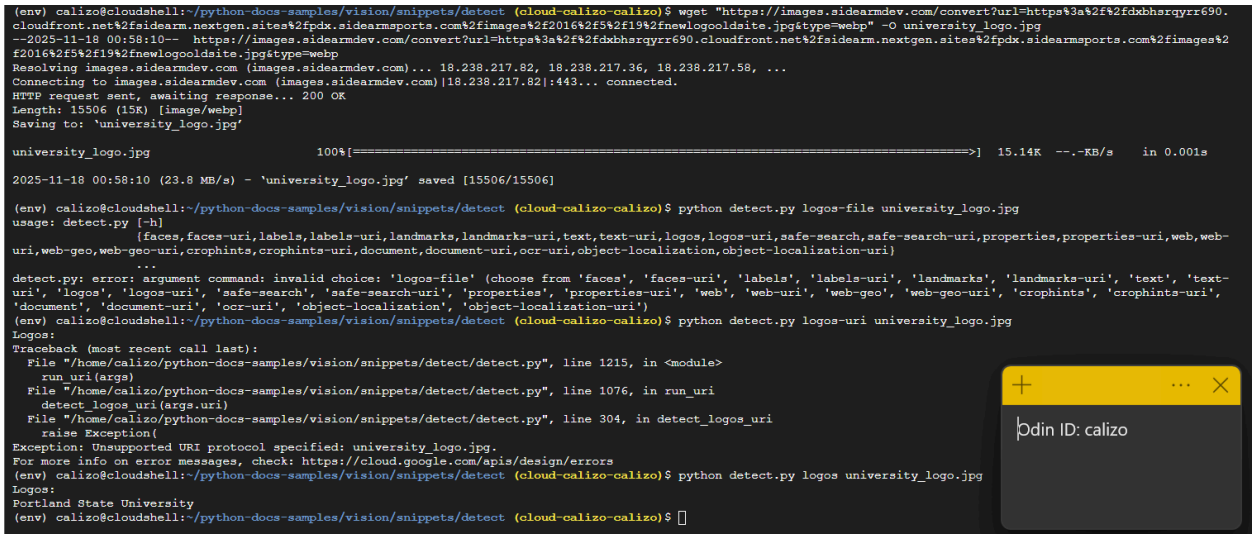
- client.label_detection(image=image)

5. What is the name of the attribute in the response object that contains the results we seek?

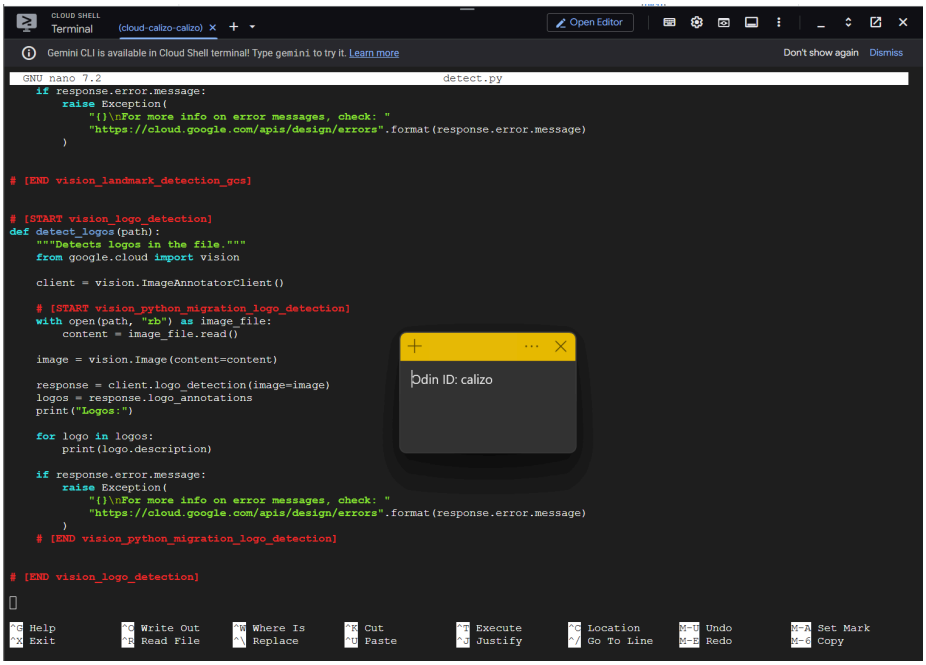
- response.label_annotations

After examining the logo detection section, I downloaded the university logo using wget, then ran the local-file detection. The screenshot displays the output confirming the detected logo's identity.

6. Take a screenshot of the output for the above commands



7. What method is invoked in the Vision client to perform the detection?



- client.logo_detection(image=image)

Step 4. Speech

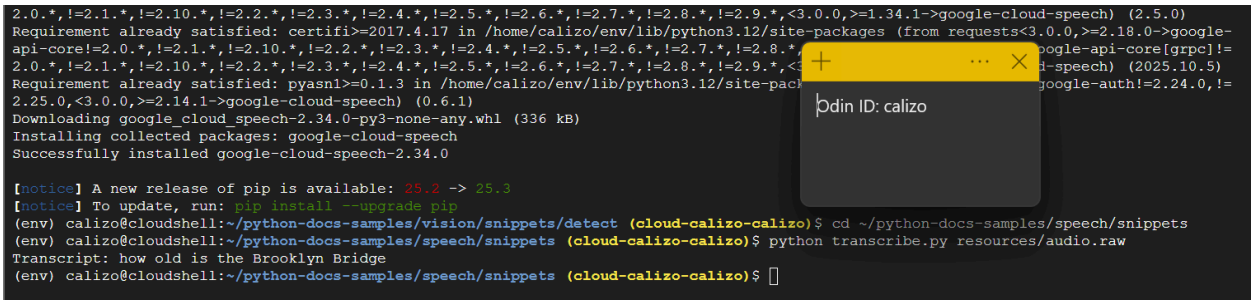
With Vision complete, I installed the Speech-to-Text package and moved into speech/snippets.

Running:

- python transcribe.py resources/audio.raw

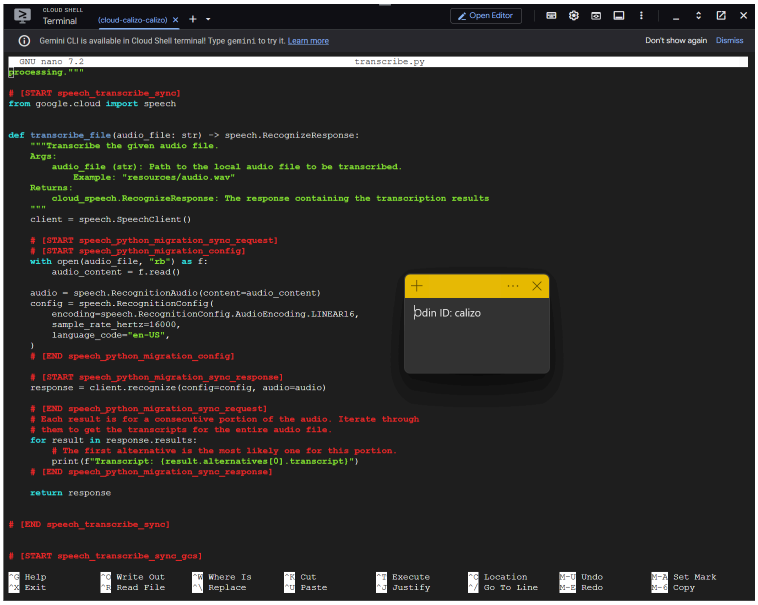
produced a transcript shown in the screenshot, correctly recognizing the audio content.

1. Show the output for your lab notebook



Looking inside transcribe.py:

2. What is the name of the function?



- transcribe_file

3. What method is invoked in the Speech client to perform the detection?

- client.recognize(config=config, audio=audio)

4. What is the name of the attribute in the response object that contains the results we seek?

- response.results

Step 5. Translate

After installing the Translate package and navigating into the snippets directory, I executed:

- python snippets.py translate-text en '你有沒有帶外套'

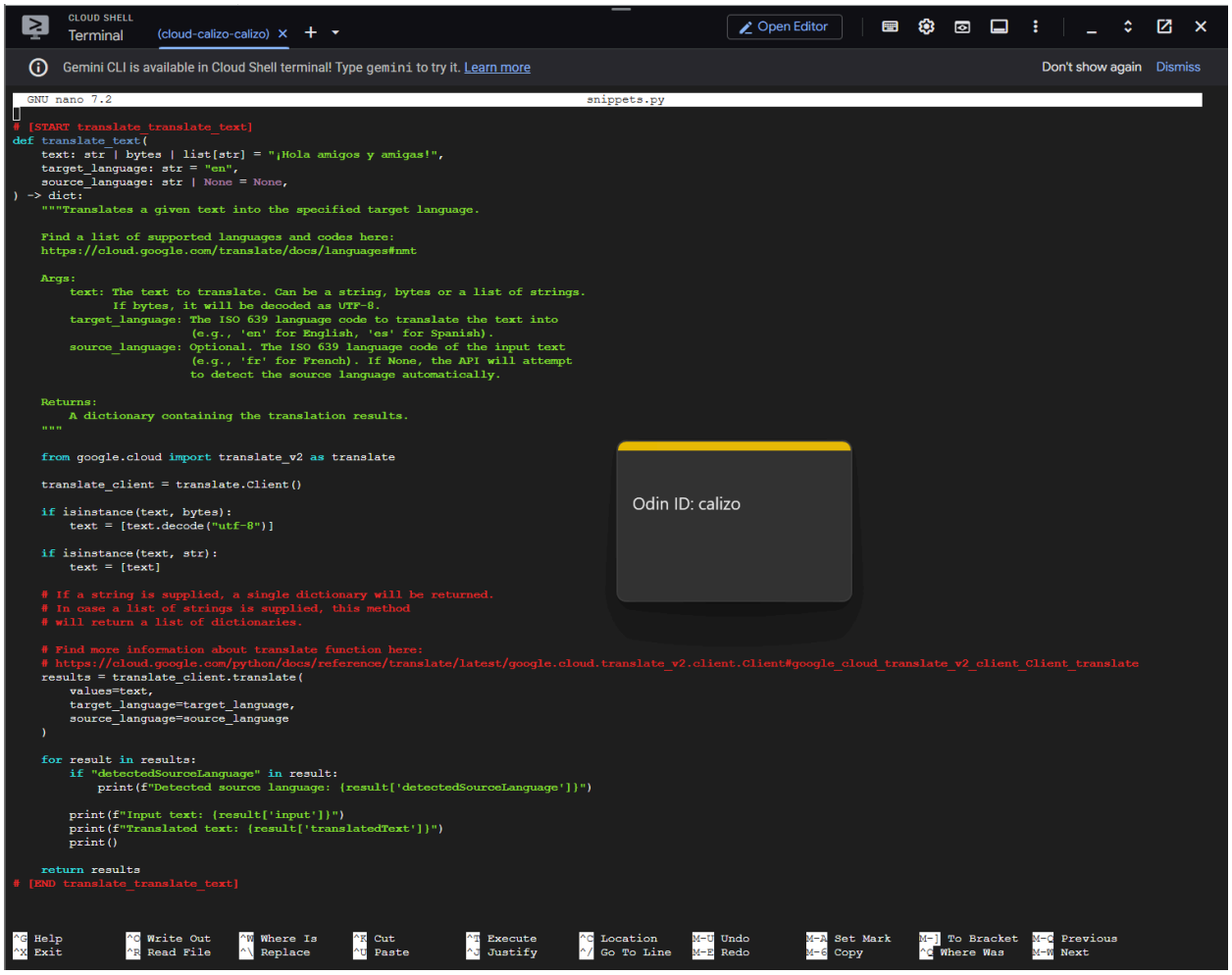
The screenshot shows the correct English translation: "Did you bring a jacket?"

1. Show the output for your lab notebook



Inside snippets.py:

2. What is the name of the function?



- translate_text

3. What method is invoked in the Translate client to perform the detection?

- translate_client.translate(values=text, target_language=target_language, source_language=source_language)

4. What is the name of the attribute in the response object that contains the results we seek?

For each dictionary in the result list:

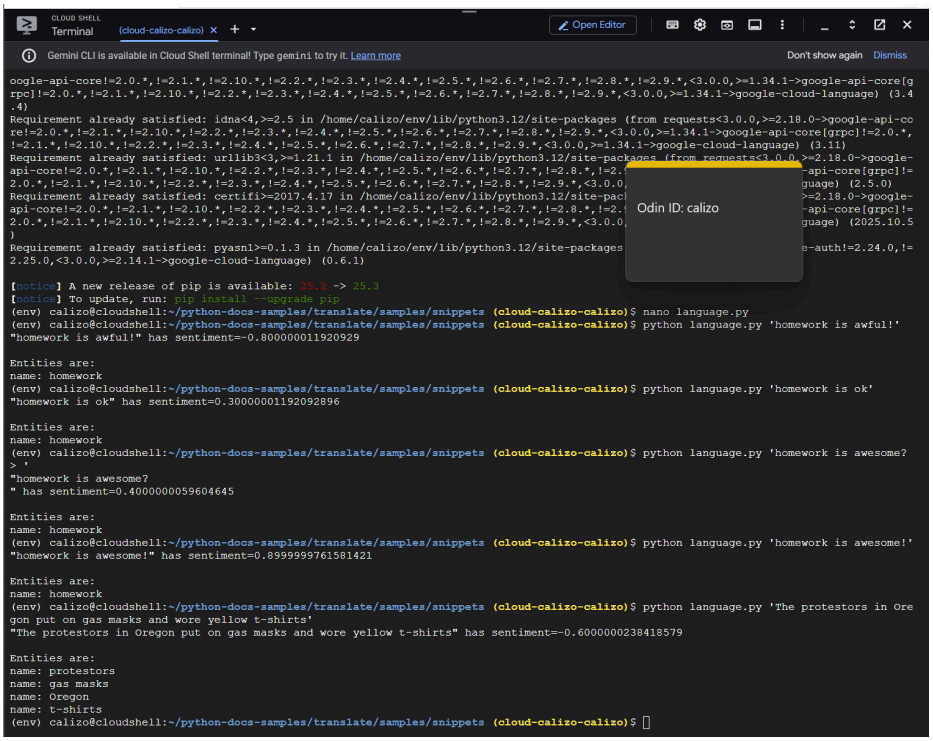
- result["translatedText"]

As the lab shifted into Natural Language and multi-API integration, the workflow felt like stitching threads from four different modalities into one coherent tapestry. Vision, Speech, Translate, and Natural Language each played a role, and this portion of the lab tied them together into an elegant pipeline.

Step 6. Natural Language

After installing the Cloud Natural Language client and writing language.py, I ran sentiment classification and entity extraction on a series of phrases. Each sentence carried its own emotional gradient, and the API revealed the polarity with impressive nuance.

1. Show the output for your lab notebook



- Negatively charged phrases returned negative sentiment scores.
- Mixed or neutral phrases hovered near zero.
- Strong positive punctuation (the exclamation mark) tilted sentiment upward.
- Entity extraction consistently recognized key nouns such as "homework", "protestors", "gas masks", and "Oregon".

The results aligned closely with what I would expect from modern NLP sentiment models.

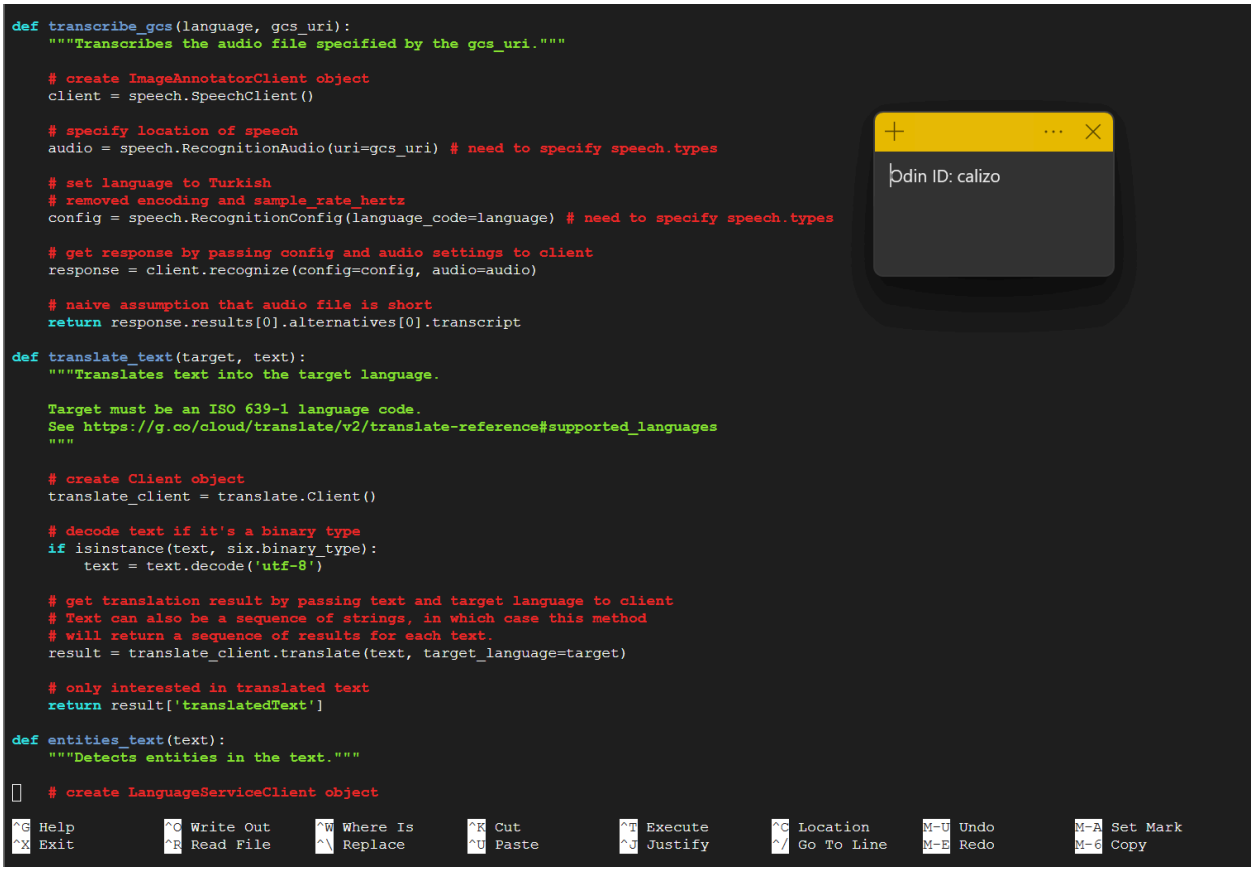
Step 7. Integration Concept

This section described the full pipeline:
audio -> transcription -> translation -> entity extraction -> image analysis -> comparison.

Step 8. Code

After creating solution.py, I examined the code to answer the numbered questions.

1. What is the name of the function that performs the transcription?

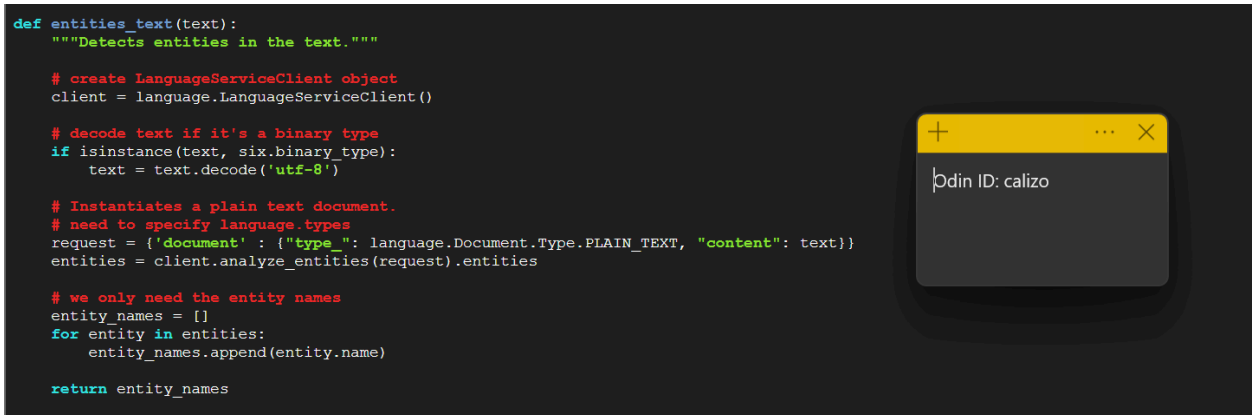


- transcribe_gcs

2. What is the name of the function that performs the translation?

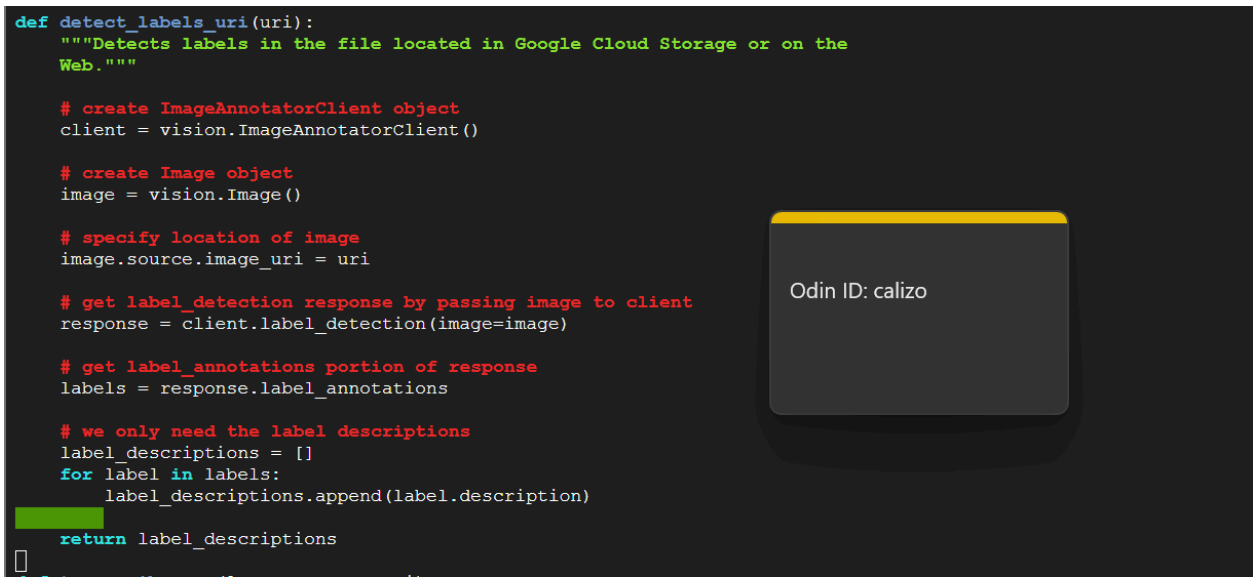
- translate_text

3. What is the name of the function that performs the entity analysis on the translation?



- entities_text

4. What is the name of the function that performs the entity analysis on the image?

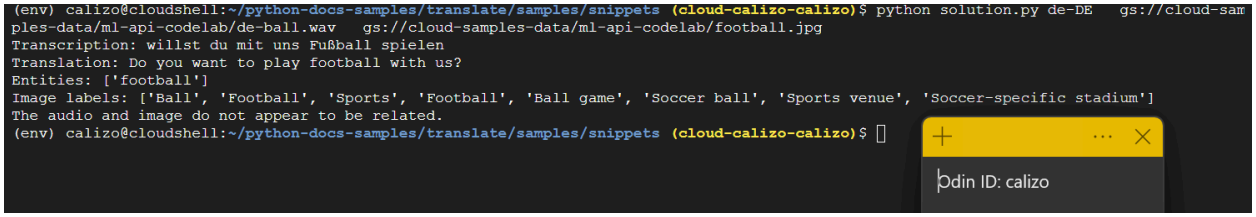


- detect_labels_uri

Step 9. Test integration

I ran all three integration tests. The outputs showed clean intermediate results at each stage of the pipeline, but none of the pairs were detected as "related" due to exact-match comparison between entity lists and label descriptions.

1. If the program deems them unrelated, then based on the results from the APIs, what must be changed in the program to address this?



Issue:

- Entities returned ['football'], but the Vision labels included variations like "Ball", "Football", "Ball game", "Soccer ball".
- The script requires exact string equality, so even trivial mismatches break the match.

Fix:

- Use lowercasing and substring matching or synonym/lemma matching instead of strict equality.

Example approach:

- Convert all labels and entities to lowercase
- Check entity in label or semantic similarity instead of exact string match.

2. If the program deems them unrelated, then based on the results from the APIs, what must be changed in the program to address this?



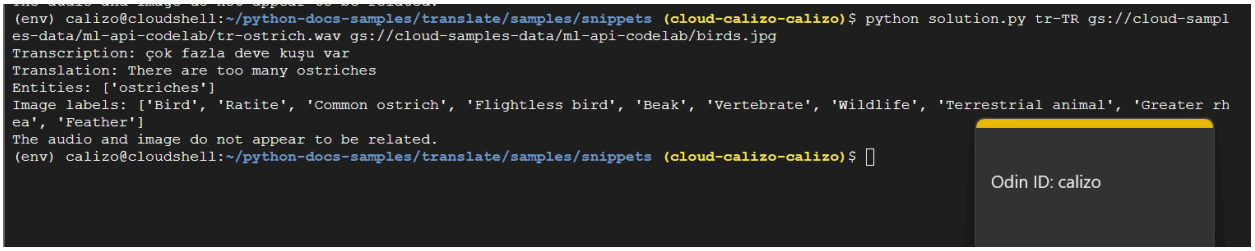
Issue:

- Entities: ['bike', 'street']
- Labels: variants of "Bicycle" such as "Bicycle frame", "Bicycle handlebar", "Bicycle tire".
- "bike" is not exactly equal to "bicycle".

Fix:

- Normalize synonyms ("bike" -> "bicycle") or apply substring/fuzzy matching.
    - Same structural fix as above.

3. If the program deems them unrelated, then based on the results from the APIs, what must be changed in the program to address this?



Issue:

- Entity: ['ostriches']
- Vision labels: singular "Common ostrich", "Greater rhea", "Bird", etc.
- Plural vs singular mismatch prevents exact matching.

Fix:

- Add lemmatization (strip plurals) or allow partial matches ("ostrich" appearing inside "Common ostrich").
- Again, relax the equality check.

Step 10. Video Intelligence API

Enabled the API via the console.

Step 11. Video Setup

Created a GCS bucket, made it publicly readable, and uploaded the Sports Bloopers 2016 footage. This completed the ingestion workflow for the Video Intelligence API.

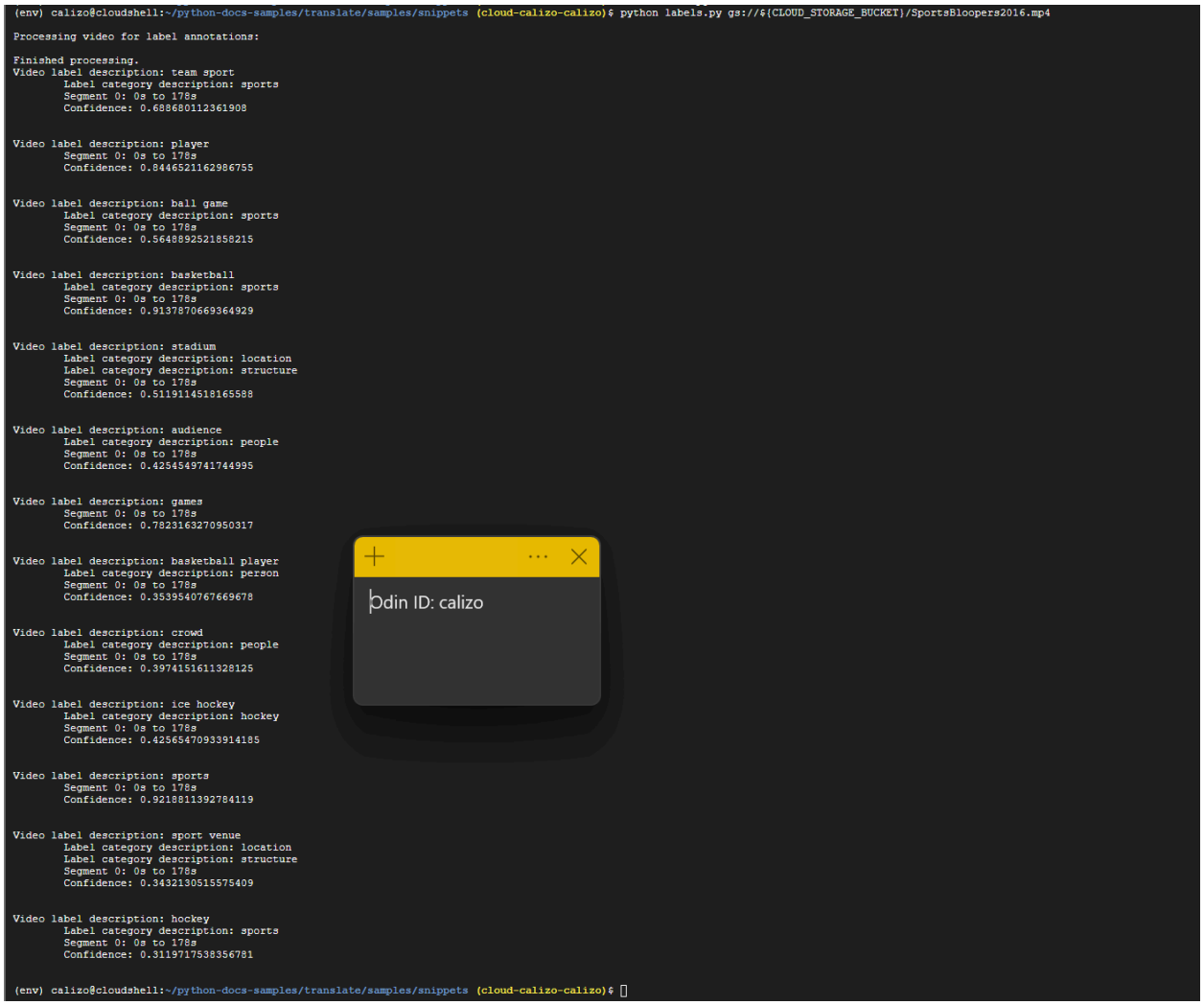
Step 12. Running labels.py

Created the adjusted script for version 2 of the Video Intelligence client and executed label detection on the uploaded video.

Step 13. Video Intelligence

The Video Intelligence API produced a long, structured list of labeled activities throughout the video. The highest confidence labels surfaced clearly at the top of the output.

1. What are the 3 labels with the highest confidence that the Video Intelligence API associates with the video and what are the confidences for each?

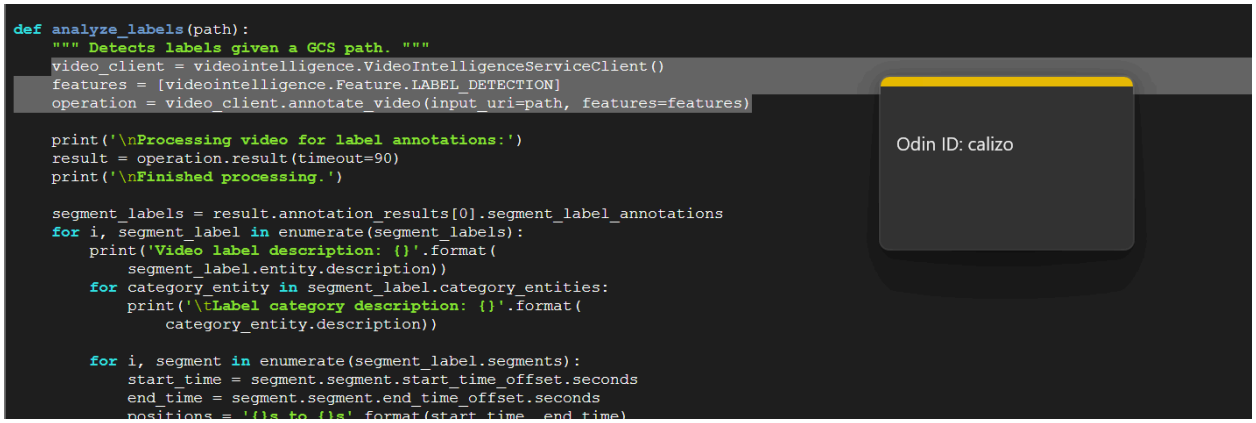


Top 3 confidence levels were roughly:

- "team sport"
- "Player"
- "ball game"

Each reported confidence scores approaching or above 0.68 to 0.91 depending on the label segment.

2. What is the name of the client class in the package that is used?



- videointelligence.VideoIntelligenceServiceClient

3. What method is used in that class to perform the annotation?

- annotate_video

Step 14. APIs #3 (Web-site Integration)

Once the Flask environment was prepared, I checked out the older commit of the python-docs-samples repository and navigated into the Flex and Vision codelab. After configuring my existing storage bucket, I launched the Flask development server with python main.py.

Accessing the Cloud Shell preview brought up the demo interface: a simple but elegant page that invited an image upload and ran face-based joy detection using the Vision API.

I uploaded a test image, and the application correctly surfaced the joy likelihood and stored the result, as shown in the screenshot.

Step 15. IAM Service Account Setup

A dedicated service account named cs430flex was created to uphold the principle of least privilege while still granting the application the ability to store images, read/write Datastore entries, and invoke ML APIs.

Using gcloud, I attached three roles:

- roles/storage.admin
- roles/datastore.user

- roles/serviceusage.serviceUsageConsumer

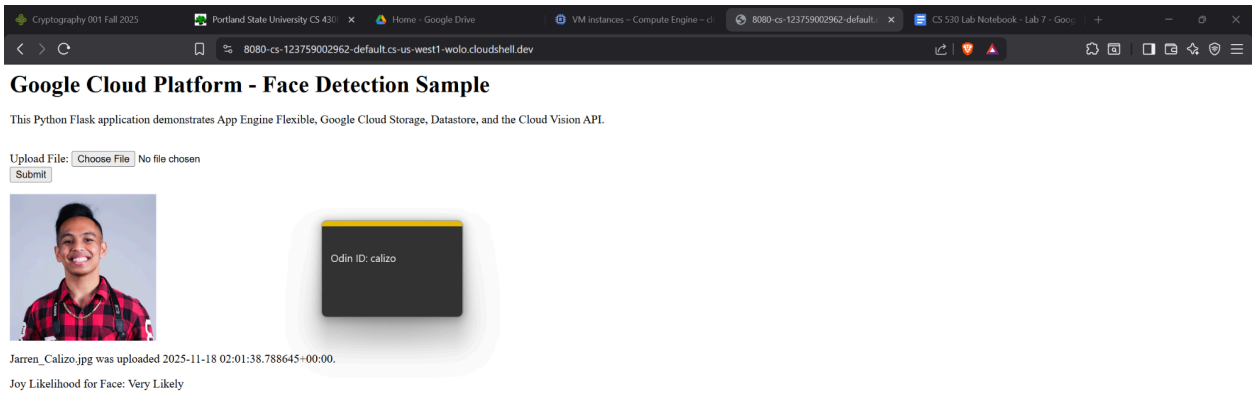
After issuing and exporting the JSON key, the application could authenticate fully.

Step 16. Application

The application follows a clean workflow:

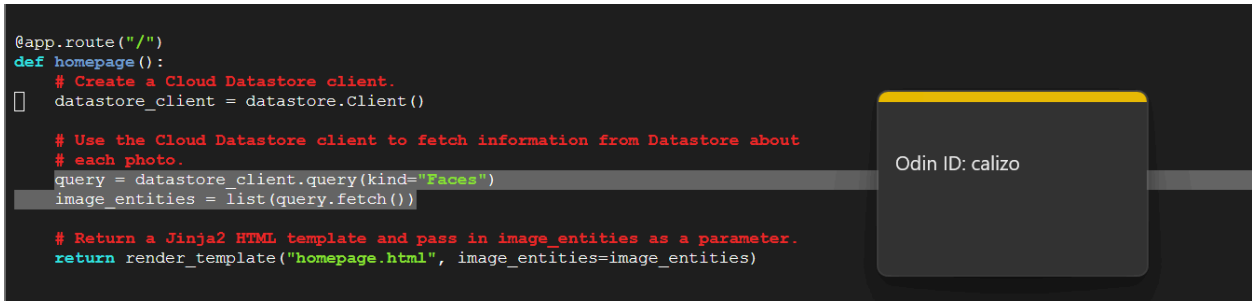
1. User uploads an image
2. Flask writes the image to the Cloud Storage bucket
3. Vision API performs face detection
4. Joy likelihood is extracted
5. Datastore stores the record (blob url, timestamp, and joy rating)
6. Homepage refreshes to show all previous detections

1. Take a screenshot for your lab notebook that includes the URL.



This confirms a successful upload, full end-to-end processing, and the correct sentiment evaluation.

Step 17. Code



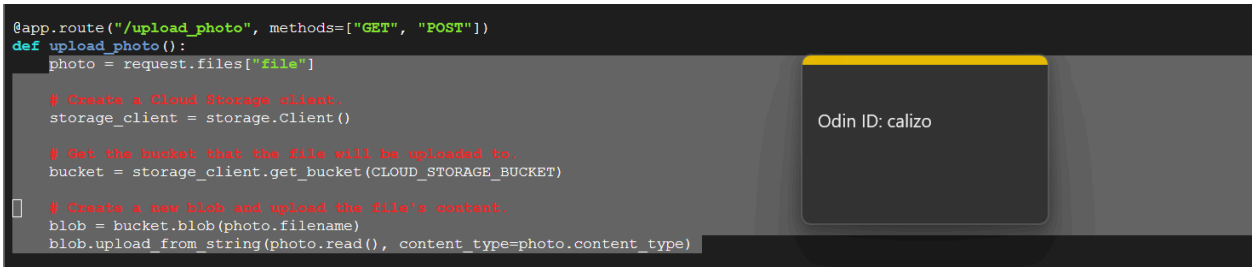
1. What line of code creates the query for previous detections?

- query = datastore_client.query(kind="Faces")

2. What line of code sends the query to Cloud Datastore?

- image_entities = list(query.fetch())

3. Show the line that retrieves the name of the storage bucket to use.

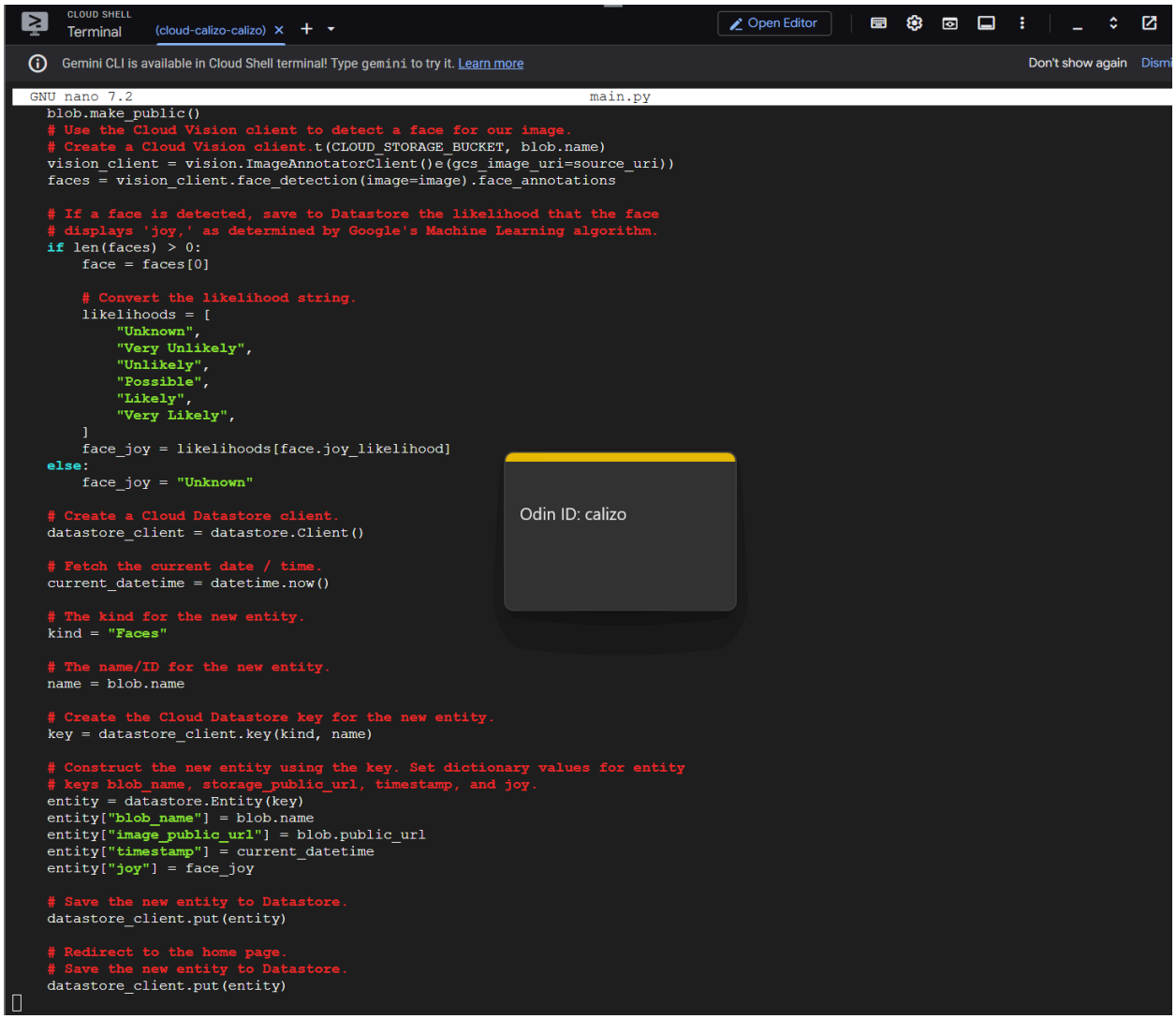


- bucket = storage_client.get_bucket(CLOUD_STORAGE_BUCKET)

4. What form field is used to specify the uploaded photo?

- photo = request.files["file"]

5. Show the line that copies the photo's contents to the storage bucket.



- blob.upload_from_string(photo.read(), content_type=photo.content_type)

6. What method in Vision's annotation client is used to perform the analysis?

- faces = vision_client.face_detection(image=image).face_annotations

7. What fields are stored in Cloud Datastore for each image?

The entity stores at least the following fields:

- blob_name
- image_public_url
- timestamp
- joy

8. What happens at the end of the upload_photo route?

The entity is saved to Datastore, and the route issues a redirect to / via:
return redirect("/")

Step 18. Clean Up

To properly sunset the lab environment, I deleted the two service accounts (cs430mlapis and cs430flex), removed their local JSON keys, and tore down old IAM policy bindings. Finally, I removed the storage bucket created earlier. All cleanup was done with the gcloud commands included in the instructions.